

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



1일차

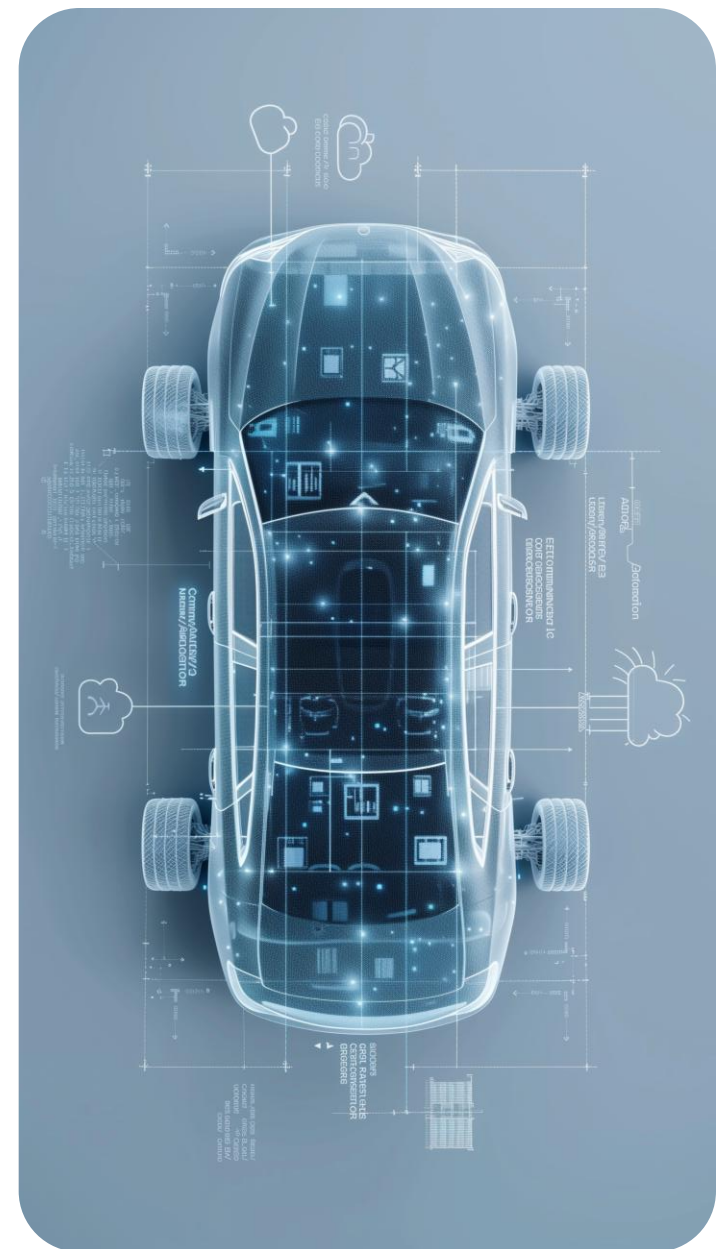
CAN 통신 기반 RTOS 제어

Telechips TOPST



Table of Contents

- 01
- 02
- 03
- 04
- 05





TOPST

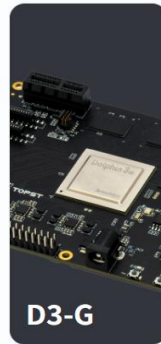
PRODUCT

TOPST offers full-featured Single Board Computers for development and production, alongside compact modules for embedded integration.

Development Boards

G Model

Powered by a field-proven SoC designed for mission-critical applications such as automotive, the G series ensures stable performance for diverse industrial, educational, and AI development needs.



D3-G



AI-G



VCP-G

Modular SoC Products

SoM Model

A compact, ready-to-integrate computing module that combines a processor, memory, and essential interfaces, designed to accelerate product development.

* SoM : (System on Module)



D5-CM

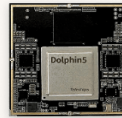
D5-CM is a compact System on Module integrating CPU, GPU, and 8TOPS-class NPU, ideal for rapid AI product development. Compatible with standard baseboards or custom carrier boards

Coming Soon ➔

SiP Model

A highly integrated chip package that combines key system components such as CPU, memory, and input/output interfaces in a compact form factor to deliver space-efficient performance. This product is intended for use in industrial solutions and will be available exclusively through professional and enterprise-level channels.

* SiP : (System in Package)



D5-SiP

AI-ready module with integrated CPU and 8TOPS NPU - optimized for edge AI and CPU processing.

Coming Soon ➔

<https://topst.ai>

<https://github.com/topst-development/Education>

Available Accessories

Industrial-Grade Accessories

This section includes both TOPST-developed accessories and verified third-party modules. All listed items are compatible with our hardware and tested for industrial and automotive use.



GSML-MIPI I/O Board

GSML-MIPI bidirectional sub-board with support for multiple virtual channels and Fakra connectors

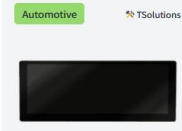
View Details



CAN Transceiver

Industrial-grade CAN interface board for real-time communication in embedded control systems

View Details



12.3" Display Board

Full-HD LVDS multi-touch panel with wide viewing angles, ideal for industrial and in-vehicle HMI

View Details



5MP MIPI CSI Camera

SMP image sensor with MIPI CSI output, suitable for AI vision tasks and camera integration on the TOPST boards.

View Details

Education Kits with Solution Partner

Academic Training Kit

While not developed by TOPST, these kits integrate our boards to provide practical, classroom-ready solutions for embedded and AI education.



D3-G EDU KIT

Learn core SoC architecture and high-performance processing using the D3-G board. This kit enables hands-on training in multicore computing, interface design, and automotive system control through realworldAI use cases like image processing and autonomous driving logic.

View Details



AI-G EDU KIT

Training kit built on the AI-G board, focusing on deep learning inference and parallel processing. Provides practical experience in vision AI and NPU-based decisionmaking, essential for autonomous driving applications.

View Details



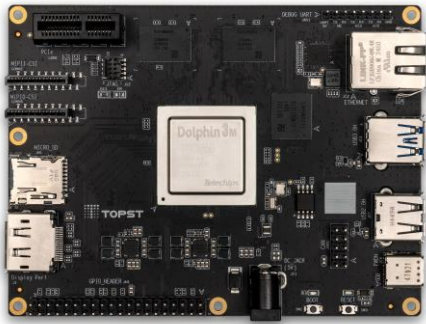
VCP-G EDU KIT

Hands-on kit for embedded system design using the VCP-G board. Covers FreeRTOS operations, realtime I/O control, and core vehicle ECU functions such as sensor integration and motor control in automotive electronics

View Details

TOPST D3-G

Description



다양한 개발을 지원하는 AP 기반 고성능 SBC

4K 디스플레이 3대를 지원하고, 여러 카메라 입력 및 실시간 2D/3D 그래픽 처리를 제공하여 산업 자동화, 스마트 키오스크, 스마트 홈 허브에 이상적인 제품입니다. 하드웨어 가상화를 통해 Hypervisor 없이도 두 개의 완벽히 분리시킨 상태에서 동시에 구동할 수 있어, 의료 기기, 보안 시스템, 고급 산업 제어 장치 등에서 뛰어난 성능을 발휘합니다.

Feature

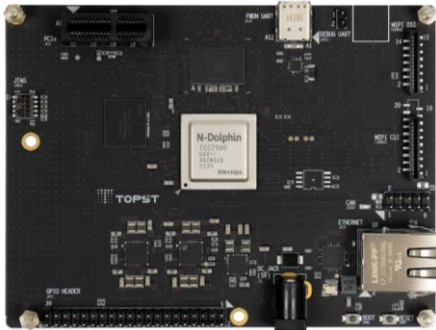
- **멀티 디스플레이 & 그래픽**
고화질 출력 및 2D/3D 프로세싱 지원
- **다채널 카메라 지원**
다중 카메라 입력 환경 지원을 위한 MIPI CSI 인터페이스
- **Hypervisor-less 솔루션**
가상화 장치 없이 다중 OS 지원
- **다양한 연결 지원**
USB 3.0, PCIe, Ethernet, 40 GPIO Pin, DP 지원
- **효율적인 프로세싱 구성**
멀티 태스킹을 위한 Cortex-A72 쿼드코어

Application

- **산업 자동화**
스마트 HMI, 로봇틱스, 다채널 카메라 분석환경
- **스마트 키오스크**
상호작용 키오스크, 디지털 메뉴 보드
- **물류**
AI 기반 디스플레이, 고객 행동분석 장비
- **보안**
감시카메라 허브, 안면인식 장비
- **스마트 홈**
멀티미디어 허브, 홈 보안 시스템

TOPST AI-G

Description



산업 전반에 걸쳐 사용 가능한 NPU 기반 엣지 비전 솔루션

첨단 비전 애플리케이션에 최적화된 고성능 NPU 기반 Edge AI 보드입니다.

ASIL-B 인증과 멀티채널 ISP 지원을 통해 뛰어난 이미지 품질과 견고한 엣지 처리 성능을 제공하며 산업 자동화, 스마트 감시 시스템, 헬스케어 분야에 적합합니다.

Feature

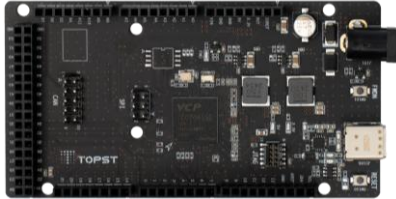
- **AI 프로세싱**
Edge 비전 응용을 위한 8 TOPS NPU
- **고속 카메라 입력**
다채널 이미지 캡처를 위한 MIPI CSI 2 인터페이스
- **고성능 출력**
MIPI DSI 및 내장 ISP 기반 FHD 디스플레이
- **퍼포먼스 효율**
실시간 처리를 위한 Cortex-A53 쿼드코어 (1.8GHz)
- **다양한 연결 지원**
PCIe, Ethernet, 40 GPIO Pin, DP 지원

Application

- **산업 자동화**
비전 기반 품질 검사, 로봇틱스
- **스마트 보안장비**
AI 기반 보안 카메라, 다채널 시스템
- **헬스케어**
이미지 분석, 환자 모니터링 시스템
- **리테일 및 물류**
자동 체크아웃 시스템, 패키지 검사 장비

TOPST VCP-G

Description



실시간 Core 기반 다양한 산업의 변화를 이끌 Cortex-R 기반 MCU

저전력, 안전 및 실시간 처리 환경에 최적화된 32비트 MCU입니다.

자동차 분야에서 시작해, 현재는 산업 자동화, 스마트 홈, 보안 시스템 등에서도 뛰어난 성능을 발휘하고 있습니다. 텔레매틱스, 클러스터 관리 기능을 갖추고 있으며, 저전력 28nm 설계, 강력한 보안(EVITA Full HSM), ASIL-B 인증을 통해 안정적인 운영을 보장합니다.

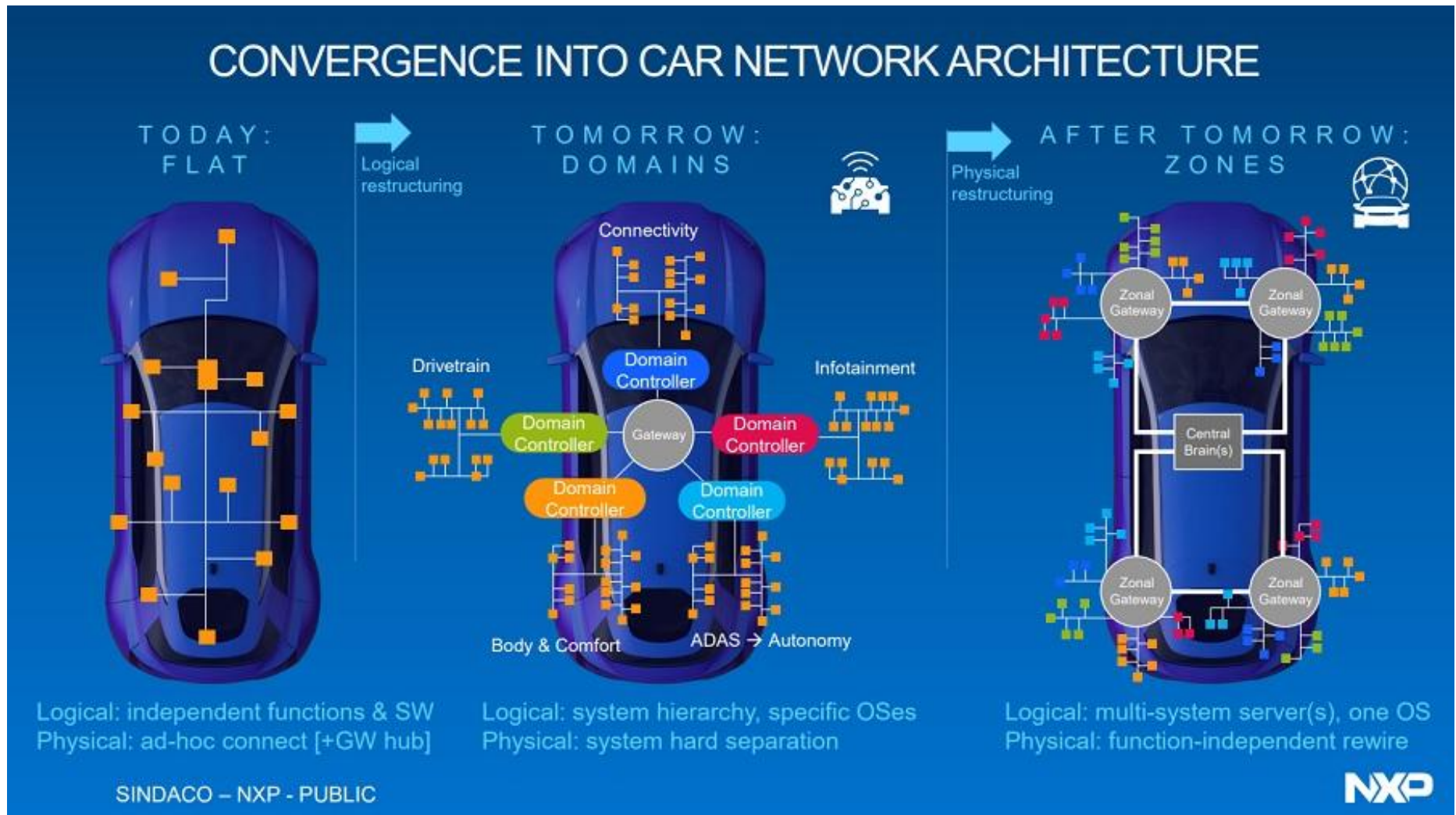
Feature

- **저전력 고성능 MCU**
Cortex-R5F 기반 32-bit MCU (4MB Flash and 512KB SRAM)
- **보안 기능**
데이터 프로세싱 보안을 위한 EVITA Full HSM
- **ASIL B 수준의 실시간 동작 성능**
텔레매틱스, 클러스터 매니지먼트
- **인터페이스 구성**
CAN, SPI, 12-bit ADC 지원

Application

- **산업 자동화**
실시간 모니터링, M2M 커뮤니케이션 보안
- **스마트 홈**
중앙 허브, 무선 충전 디바이스
- **보안 시스템**
접근 권한 관리, 보안카메라 허브
- **소비자 전자장치**
AI 기반 스마트 카메라 (제어)

Zonal Architecture

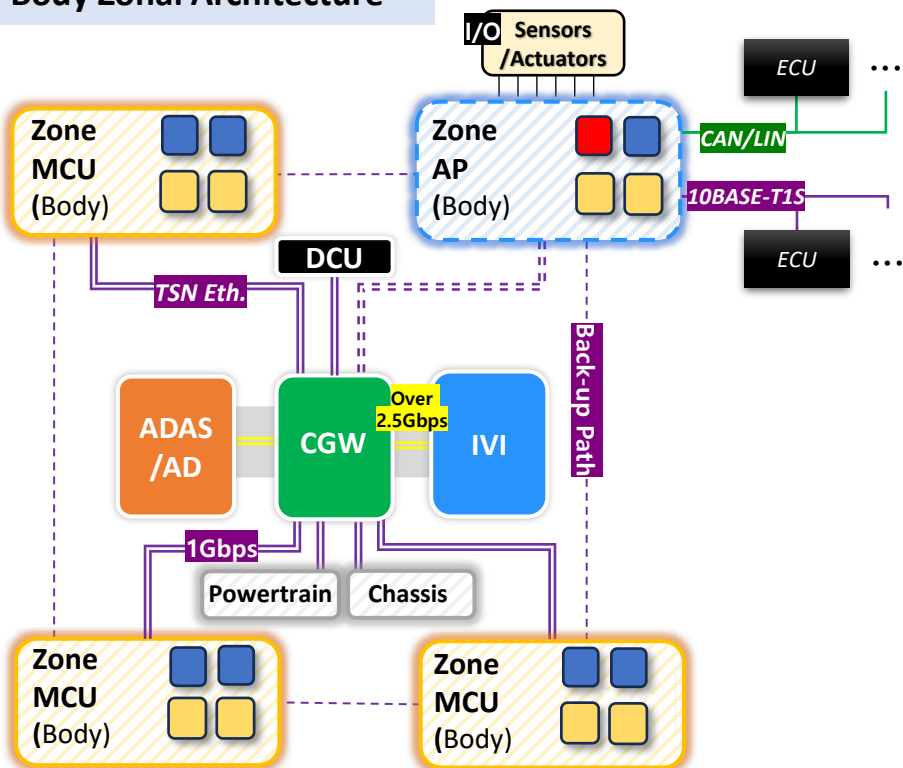


Zonal Architecture

E/E Architecture

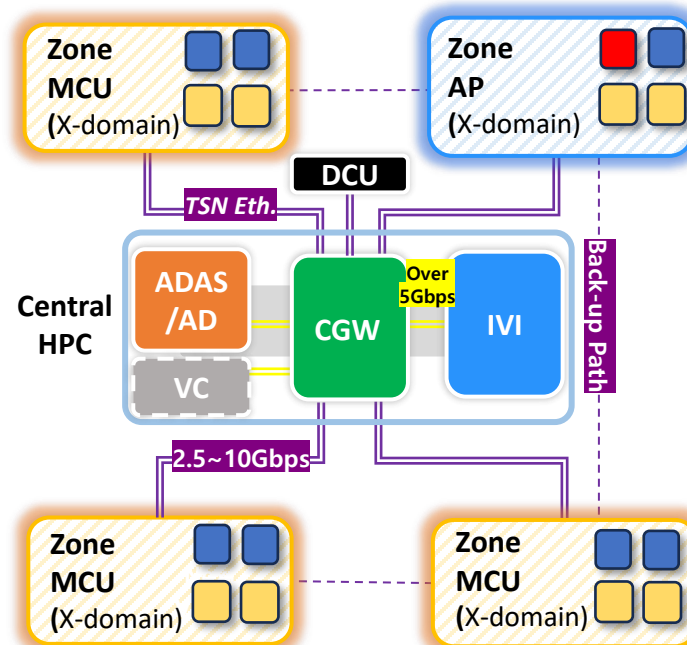
(Electrical/Electronic)

Body Zonal Architecture



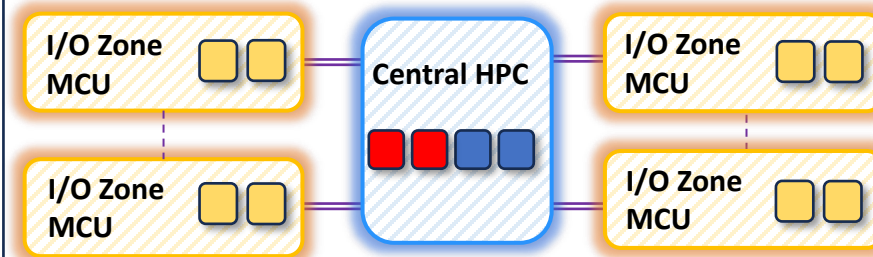
- : [MCU] Realtime Computation
- : [AP] Complex/High performance Computation
- : Aggregated I/O Control & G/W

Cross-Domain Zonal Architecture



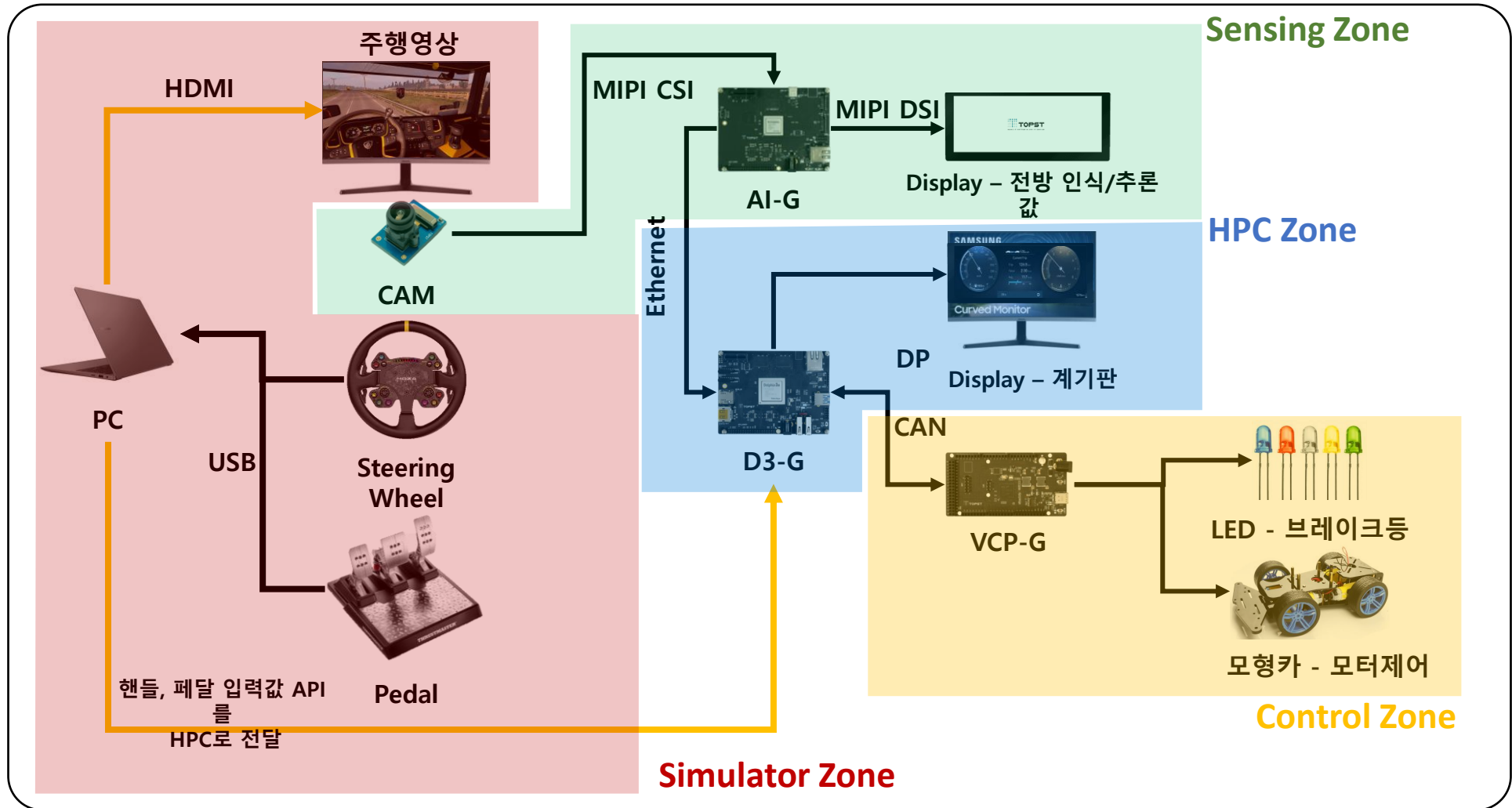
각 Zone controller가 body, chassis, power train 제어 기능을 통합

Central Computer - (I/O) Zonal Architecture



분산된 domain 제어 기능을 고성능 Central HPC로 통합,
zone controller는 I/O aggregation 기능

Zonal 기반 차량 인터랙션 시스템 구현



수업 계획

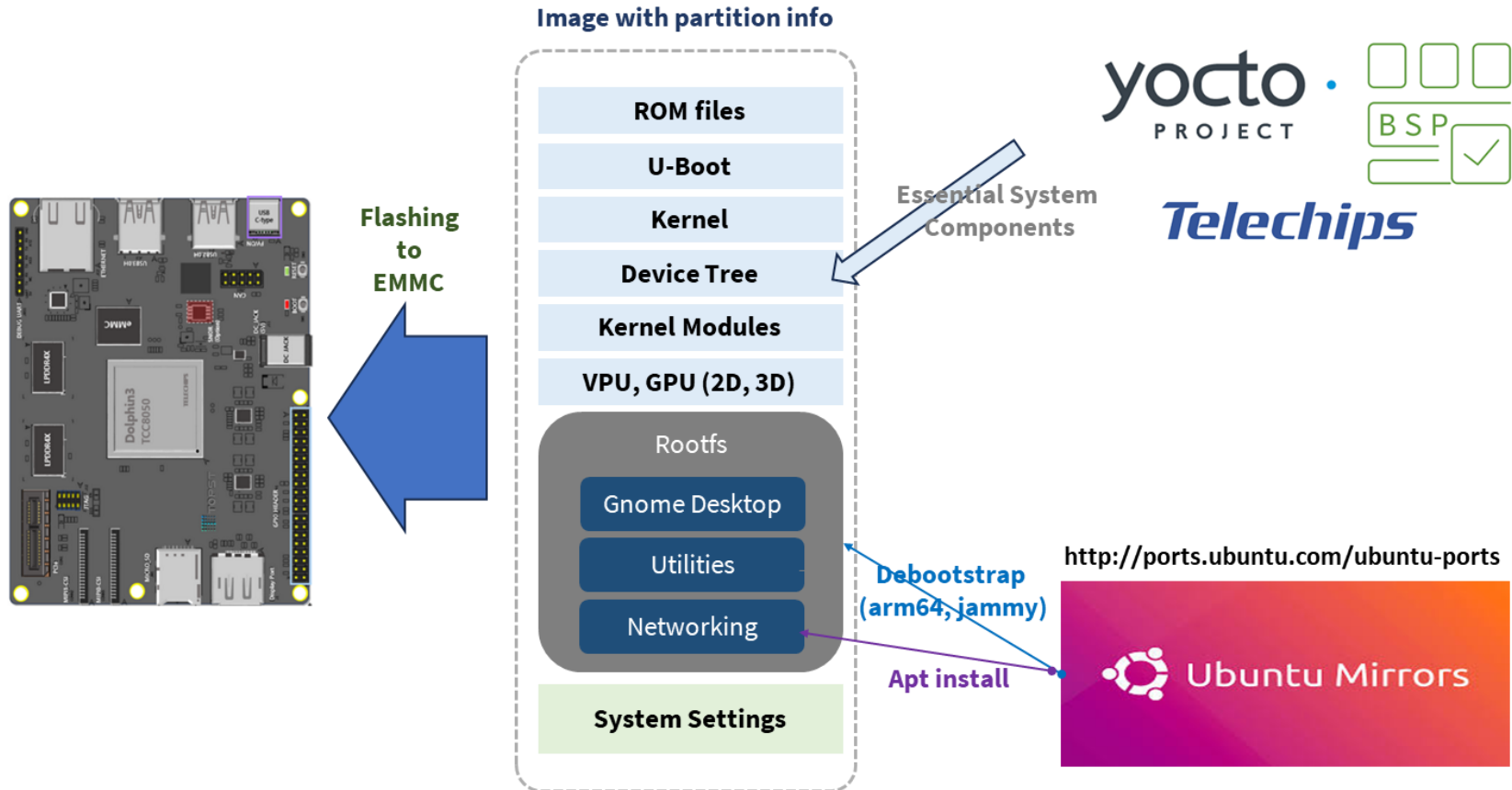
일자	1교시	2교시	3교시	4교시	5교시	6교시
1일차	Introduction	실습 환경 세팅	TOPST D3-G 환경 세팅	TOPST VCP-G 환경 세팅	CAN 기반 통신 실습	
2일차	NPU 모델 변환 개요	NPU 모델 변환 및 추론 실습 (yolov8s & UFLD)		QT5 기반 GUI 구현 및 실습	SDV 통합 시스템 구현 및 실습	

D3-G Specification

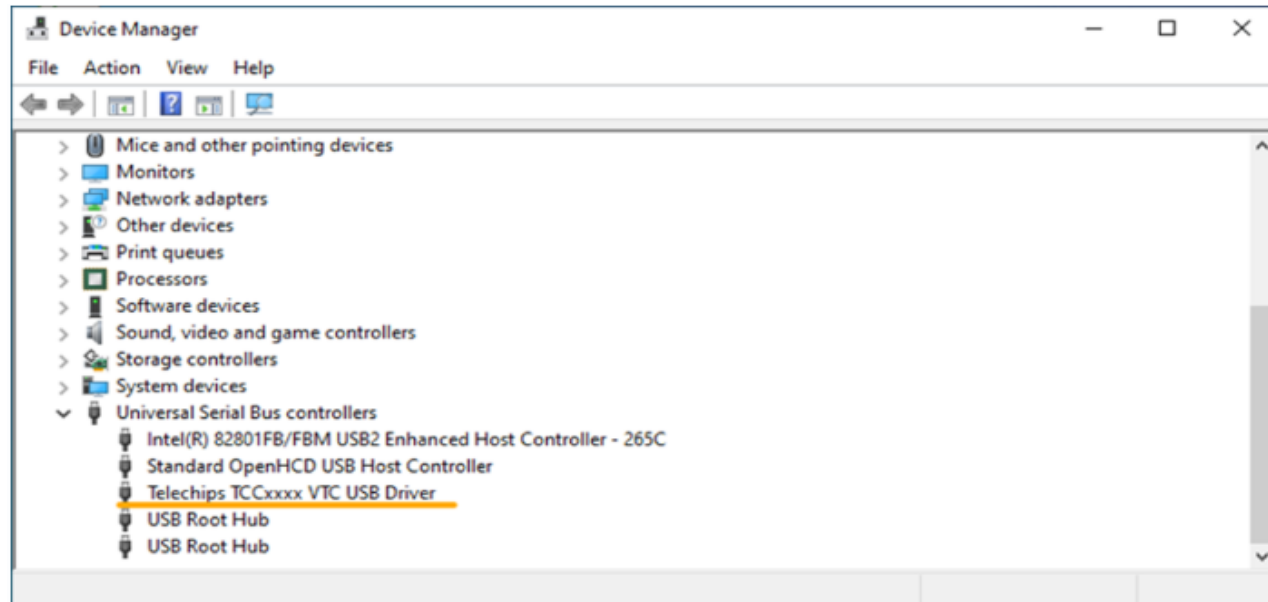
Part Name	D3-G
CPU	Arm® Cortex-A72 1.6 GHz, Quad
CPU	Arm® Cortex-A53 1.45 GHz, Quad
CPU	Arm® Cortex-R5
GPU	168 GFLOPS, IMG PowerVR 9XTP
Memory	4 GB (2 GB 32-bit LPDDR4 x 2)
Storage	microSD Card (Not included), 8 GB eMMC 5.1 Flash Storage
Automotive Network	CAN 3-channel
Network	Gigabit Ethernet
Camera	MIPI CSI-2 2-lane, MIPI CSI-2 4-lane
Display	DisplayPort 1.4
USB	USB 2.0 Host, USB 2.0 DRD, USB 3.0 DRD
Others	PCIe Gen3, UART/JTAG debugger port, I2S, I2C, SPI, PWM
Power	5V/3A
OS	Linux
Mechanical	120 mm x 90 mm

- D3-G 는 3개의 코어 존재
 - Cortex-A72
 - Cortex-A53
 - Cortex-R5
- 본 실습에선 **A72 코어와 R5 코어만**을 사용
 - A72 – Ubuntu Weston Desktop 설치
 - R5 – FreeRTOS 설치

D3-G Ubuntu 이미지



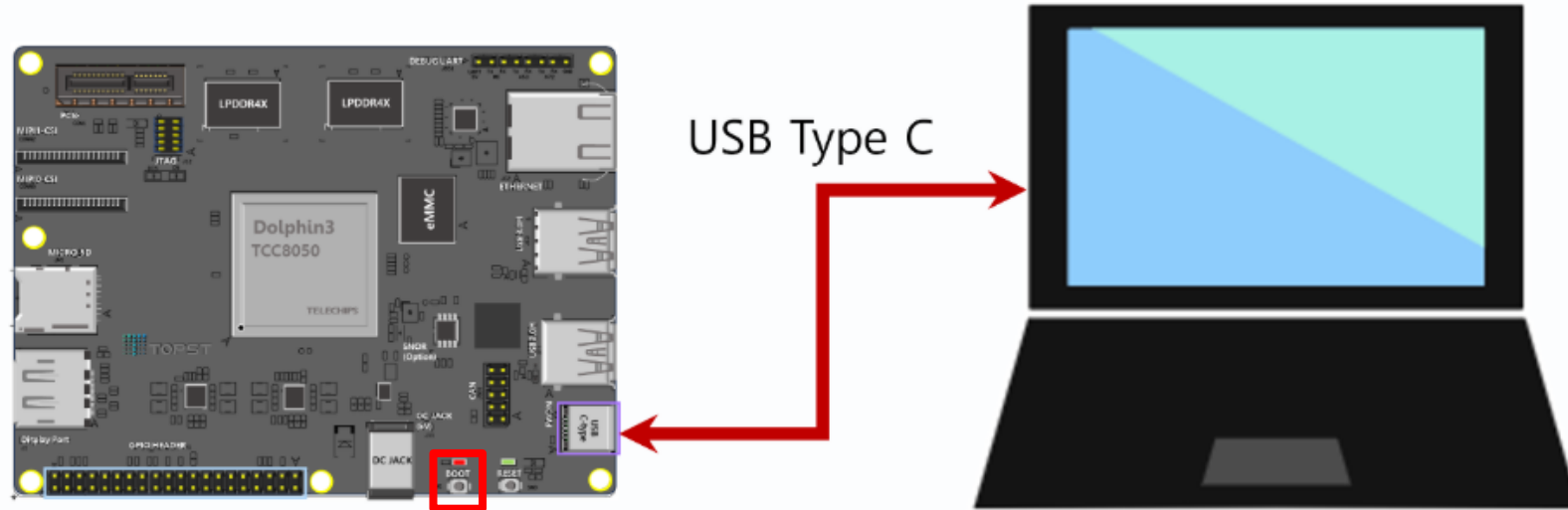
D3-G 환경 설정(공통)



• Install VTC Driver

- 펌웨어 다운로드를 위해 VTC(Vendor Telechips Certification) 드라이버 설치 필수
- 관리자 권한으로 실행: [VTC 드라이버](#)
- 드라이버 설치 후 FWDN 모드에서 USB 연결 후 설정 확인

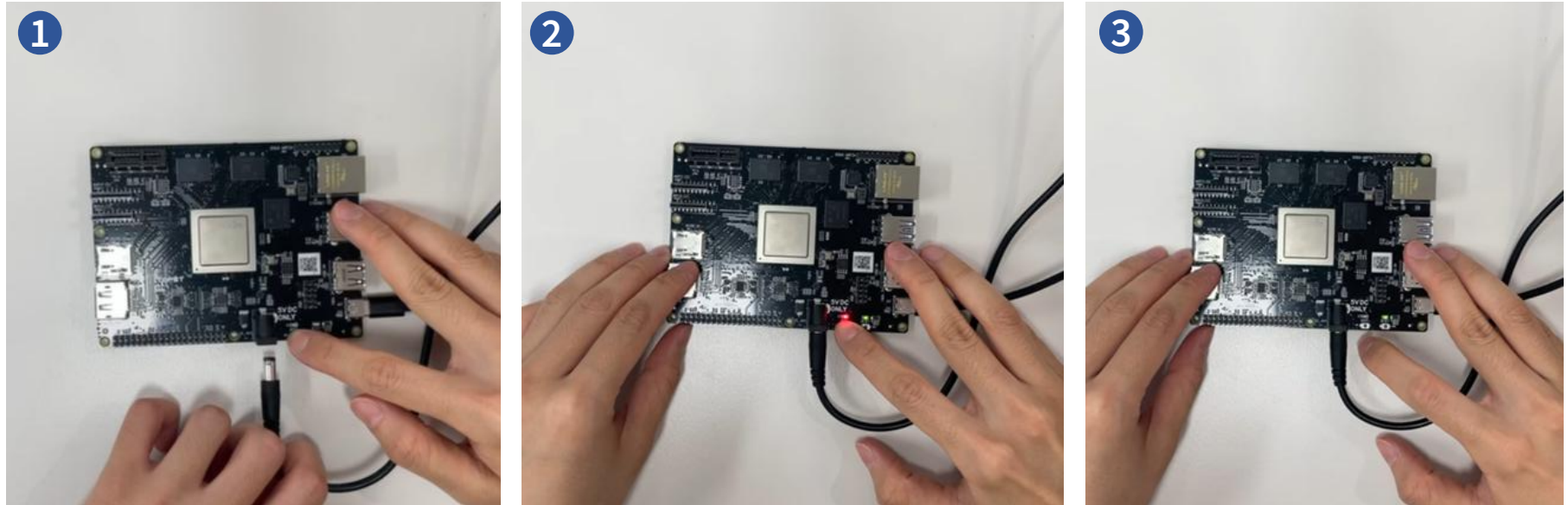
D3-G 환경 설정 (공통)



- **Connect D3-G Board and Host PC with USB Boot Mode**

- USB Boot Mode로 들어가기 위해 FWDN 스위치를 누른 상태에서 전원 케이블을 D3-G 보드에 연결
- USB Type-C 케이블을 D3-G 보드 및 Host PC의 USB Type-C FWDN 포트에 연결
- D3-G : **C 케이블** & Host PC : **USB 케이블**

D3-G 환경 설정 (공통)



- **Boot Mode 진입**

- BOOT 스위치를 누른 채로, 전원 플러그 연결(BOOT Switch LED 상태 : ON)
- 이후 스위치에서 손가락을 떼면 진입 완료(BOOT Switch LED 상태 : OFF)

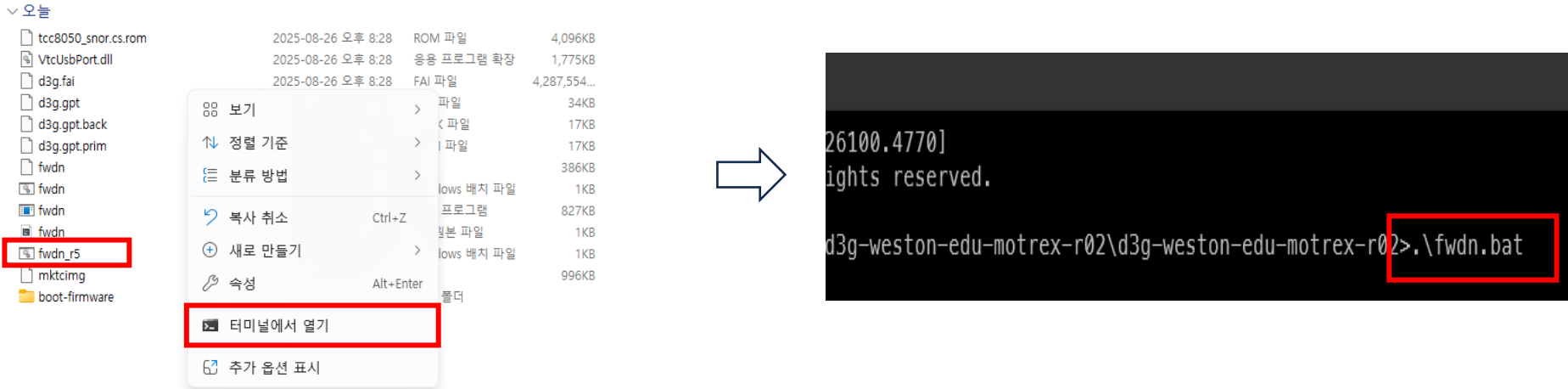
D3-G A72 이미지 업로드 (공통)



• FWDN (D3-G Ubuntu Weston/Gnome Desktop)

- 제공된 d3g-ubuntu zip 파일을 PC Windows 혹은 PC ubuntu에 다운로드 후 압축 해제
- Ex) Windows
 - 받은 폴더 내에서 마우스 우클릭> 터미널에서 열기(명령프롬프트) 실행
 - fwdn.bat (Windows 배치 파일)실행
 - fwdn.bat은 펌웨어를 자동으로 다운로드 하는 실행 파일
 - \$.\fwdn.bat

D3-G R5 이미지 업로드 (공통)

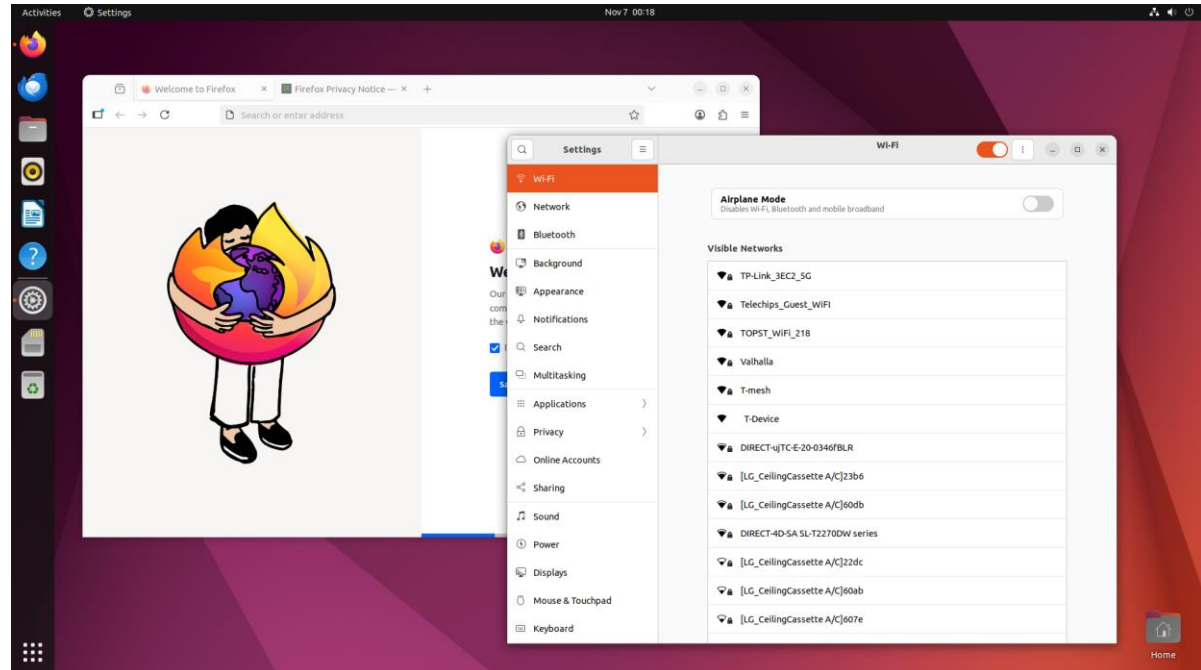


• FWDN (D3-G Ubuntu Weston/Gnome Desktop)

- 제공된 d3g-ubuntu zip 파일을 PC Windows 혹은 PC ubuntu에 다운로드 후 압축 해제
- Ex) Windows
 - 받은 폴더 내에서 마우스 우클릭> 터미널에서 열기(명령프롬프트) 실행
 - fwdn_r5.bat (Windows 배치 파일)실행
 - fwdn_r5.bat은 펌웨어를 자동으로 다운로드 하는 실행 파일
 - \$.\fwdn_r5.bat

D3-G A72 이미지 (Yocto Weston)

- D3-G 보드에 모니터와 usb 허브 및 키보드, 마우스, wifi 동글을 연결하고 재부팅(reset 버튼 or 전원 재인가)
- Ubuntu 로그인창에서 암호 : topst 입력
- Gnome desktop 창 확인



Demo Web-site on localhost

COG browser

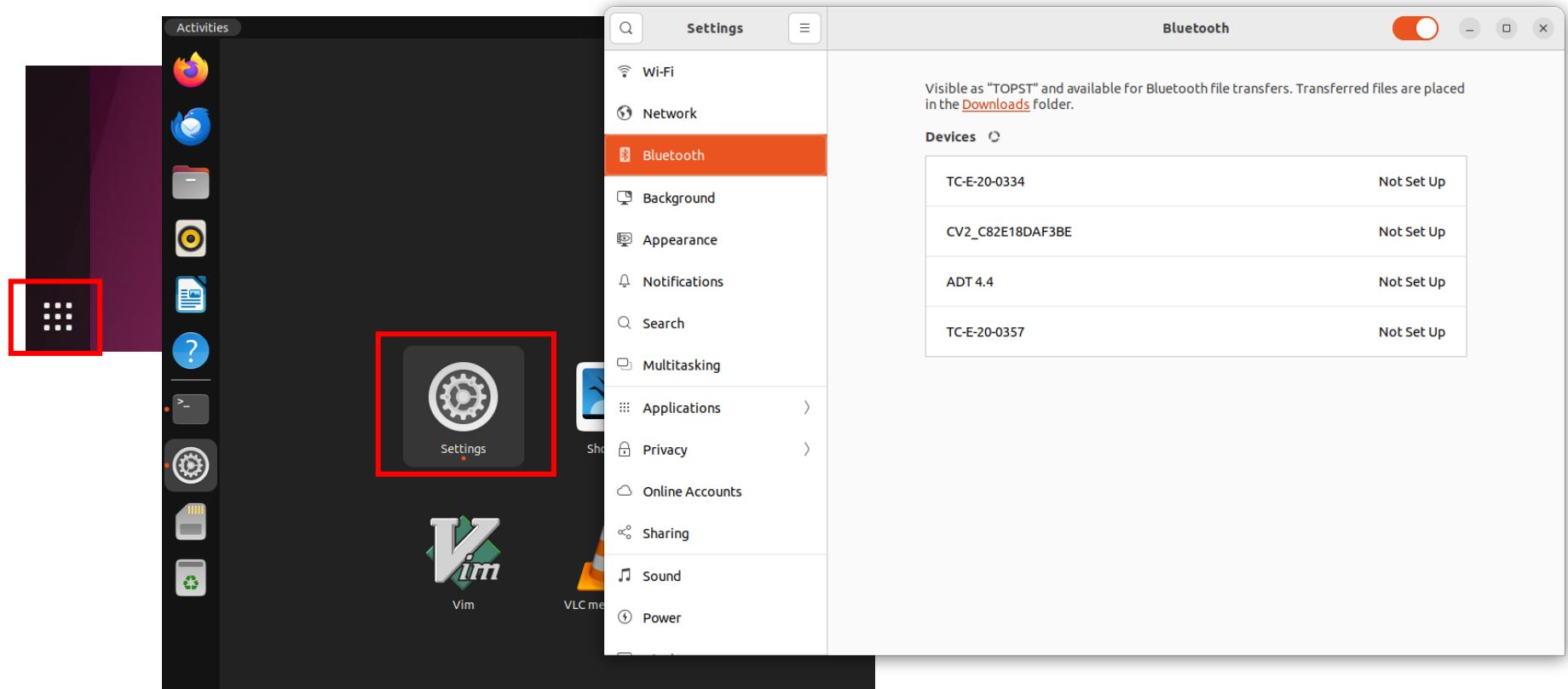
WPE Webkit

WPE Backend

D3-G system

D3-G Ubuntu 환경 설정

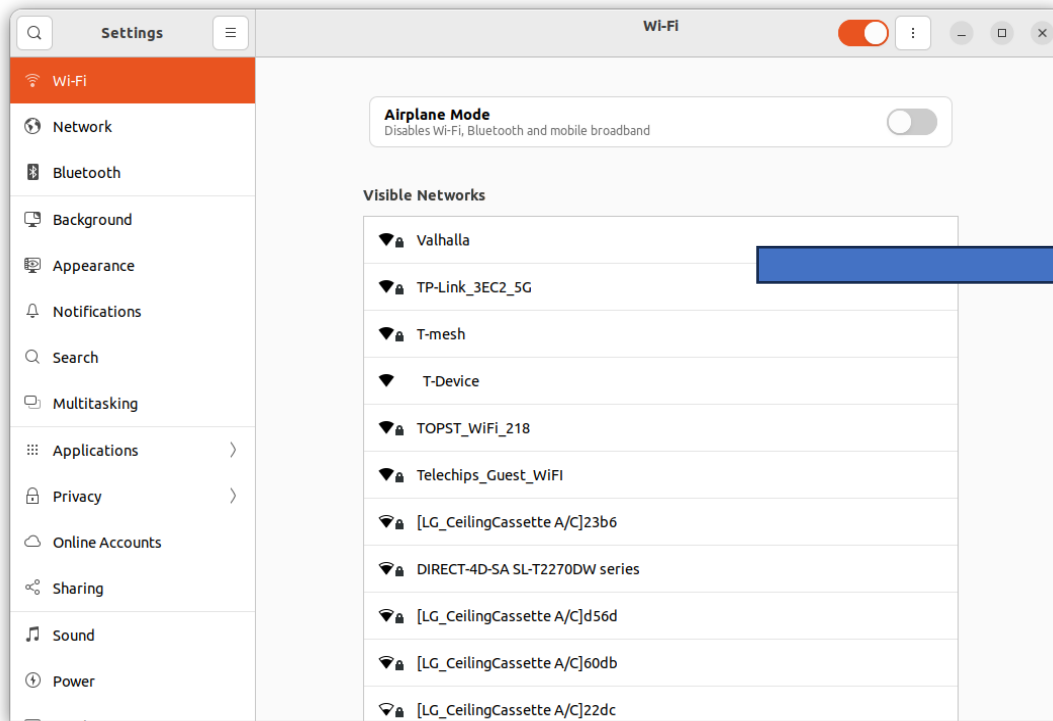
- **Settings 실행**
 - Network, Sound, Bluetooth 등 전반적인 시스템 설정



D3-G Ubuntu 환경 설정

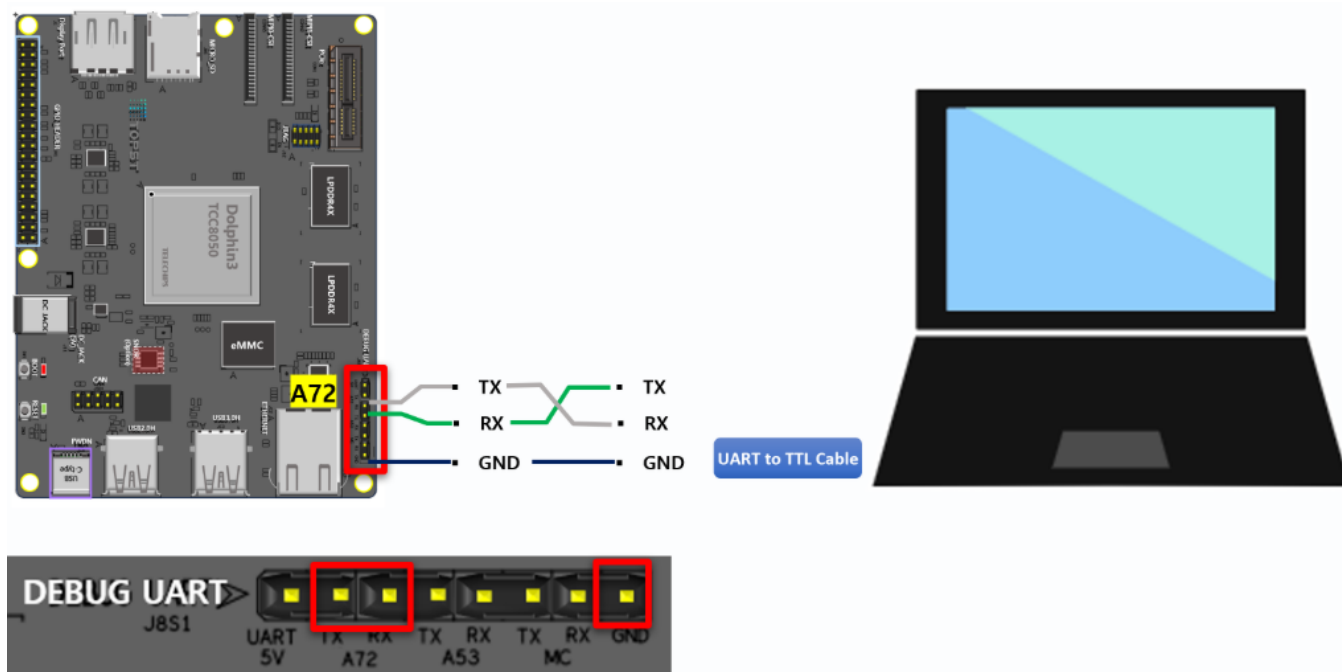
- 네트워크 설정

- Settings에서 Wifi 설정 진행 : AP 선택하고 비밀번호를 입력해 인터넷 연결



선택해서 연결

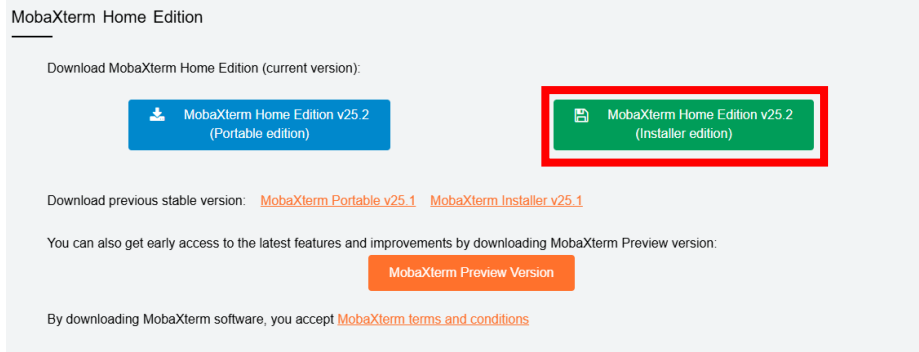
D3-G Ubuntu 환경 설정



- **Connect D3-G A72 and Host PC with UART**

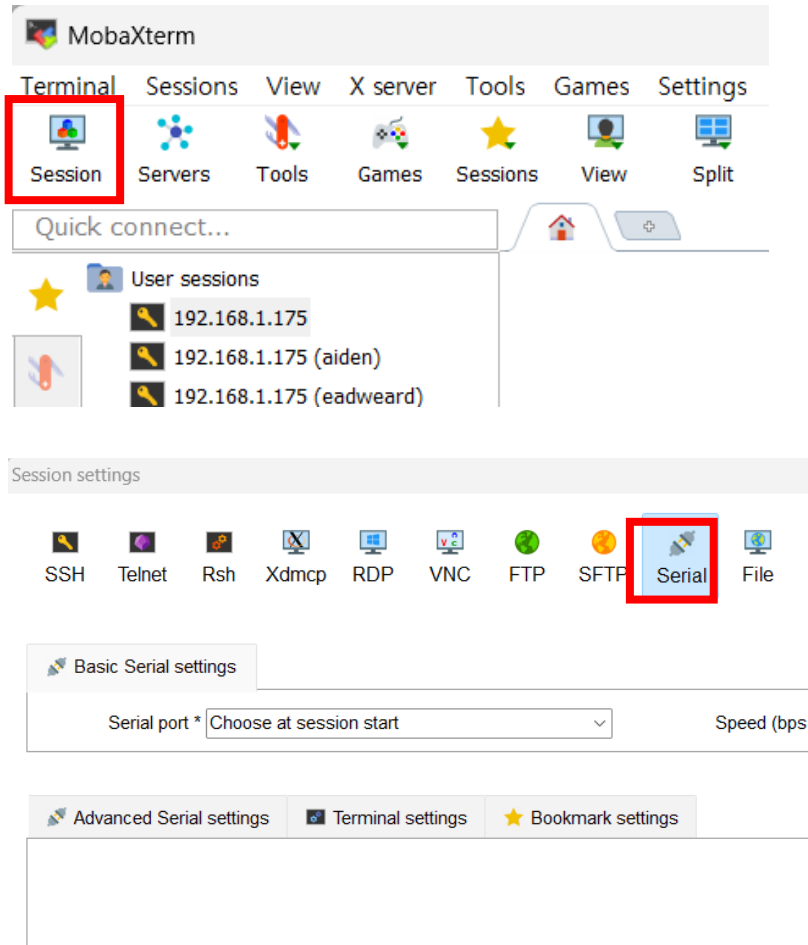
- D3-G A72 코어와 PC 간 USB-to-TTL cable을 이용하여 연결
 - 검정 케이블은 UART 핀의 **GND**에 연결
 - 흰 케이블은 UART 핀의 **TX** 핀에 연결 & 녹색 케이블은 UART 핀의 **RX** 핀에 연결

D3-G Ubuntu 환경 설정

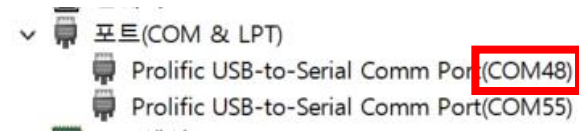


- **Connect D3-G Board and Host PC with UART**
 - Serial port driver 설치
 - [CP210x Universal Windows Driver](#)
 - [PL2303_prolific Driver](#)
 - **Terminal emulator 설치 (MobaXterm)**
 - [MobaXterm Xserver with SSH, telnet, RDP, VNC and X11 - Download](#) - 링크 접속
 - **압축 해제**
 - 설치한 MobaXterm_Installer_v25.2 압축 해제
 - MobaXterm_installer_25.2 실행파일 실행
- Next 두는 우 실시 진행

D3-G Ubuntu 환경 설정



- 윈도우 검색창에 MobaXterm 검색 후 실행
- 오른쪽 상단에 Session 버튼 클릭
- Serial 클릭
- 장치 관리자를 통해 port 확인



D3-G Ubuntu 환경 설정

Basic Serial settings

Serial port * COM4 (Prolific USB-to-Serial Comm Port(CO... Speed (bps) * 115200

Advanced Serial settings Terminal settings Bookmark settings

Serial engine: PuTTY (allows manual COM port setting)

Data bits 8

Stop bits 1

Parity None

Flow control None

Reset defaults

Execute macro at session start: <none>

If you need to transfer files (e.g. router configuration file), you can use MobaXterm embedded TFTP server

"Servers" window --> TFTP server

- Serial port에서 연결된 포트 설정
 - **Speed 115200** 설정
- Advanced Serial settings 탭 선택
 - **Flow control**에서 **None** 선택
- ok눌러 보드 접속
- ID : topst / PW : topst 입력

D3-G Ubuntu 환경 설정

- D3-G A72로 부팅한 콘솔화면에서 다음과 같은 커맨드로 파티션을 확장할 수 있다.

- `$ parted`
- `$ rescue -> Fix -> 0 -> 100%`
- `$ p -> resizepart 4 -> Yes -> 100%`
- `$ quit`
- `$ sudo resize2fs /dev/mmcblk0p4`
- `$ df -h`

```
root@TOPST:~# parted
GNU Parted 3.4
Using /dev/mmcblk0
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) rescue
Start? 0
End? 100%
(parted) resizepart 4 100%
Warning: Partition /dev/mmcblk0p4 is being used. Are you sure you want to
continue?
Yes/No? Yes
(parted) quit
Information: You may need to update /etc/fstab.

root@TOPST:~# resize2fs /dev/mmcblk0p4
resize2fs 1.46.5 (30-Dec-2021)
The filesystem is already 30471956 (1k) blocks long. Nothing to do!
```

FreeRTOS



- **FreeRTOS(Free Real-Time Operate System)**

- 임베디드용 오픈소스 RTOS
- MCU(MicroController Unit) 환경에서 멀티태스킹, 스케줄링, 동기화, 시간제어 기능을 제공
 - 멀티태스킹 : 여러 작업(Task)을 동시에 실행하도록 관리
 - 스케줄링 : 각 작업의 **실행 순서와 우선순위를** 정함
 - 동기화 : 여러 작업이 **공유 자원을 충돌 없이** 사용
 - 시간제어 : 작업을 **지연, 주기적 실행, 타임아웃** 등으로 제어
- 커널 크기가 가볍고 빠름
- 사용 예
 - 산업 : 공장 자동화 정비, 로봇 제어기 등
 - 자동차 전장 : 조명/모터 컨트롤러, 와이퍼 등
 - IoT 디바이스 : 전등 스위치, 드론, 온도 센서 등

FreeRTOS의 이해

- RTOS (Real-Time Operating System)
 - 정해진 시간 내에 반응해야 하는 시스템
 - 자동차 ECU, 모터 제어, 센서 수집, 통신 스택 등과 같은 실시간성 기능군
- 주요 기능

기능	설명
태스크(Task)	병렬로 동작하는 여러 실행 단위
스케줄러(Scheduler)	어떤 태스크가 CPU를 사용할지 결정
시간 관리(Timer)	주기적 타이머 인터럽트를 통해 태스크 전환
동기화(Semaphore, Mutex)	자원 공유 시 충돌 방지
통신(Queue, Event Group)	태스크 간 데이터 교환

VCP-G Software Architecture

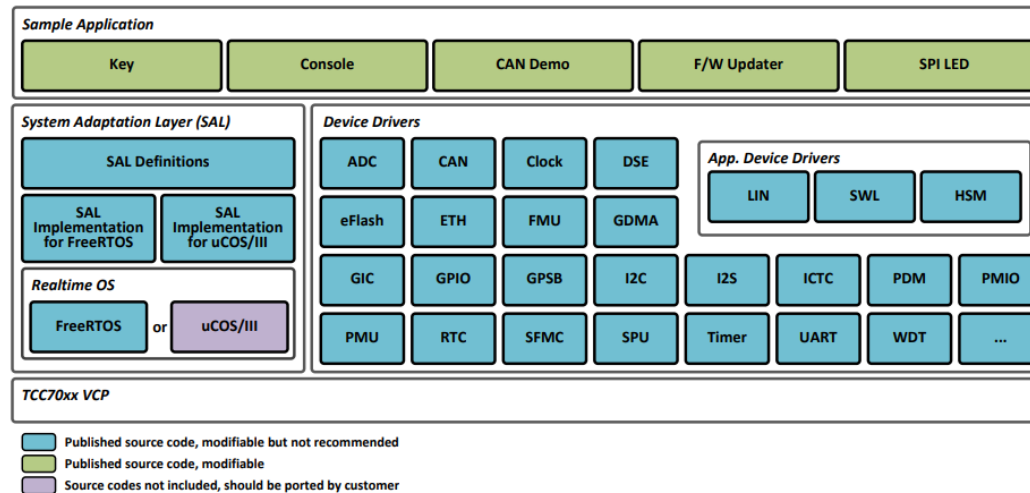
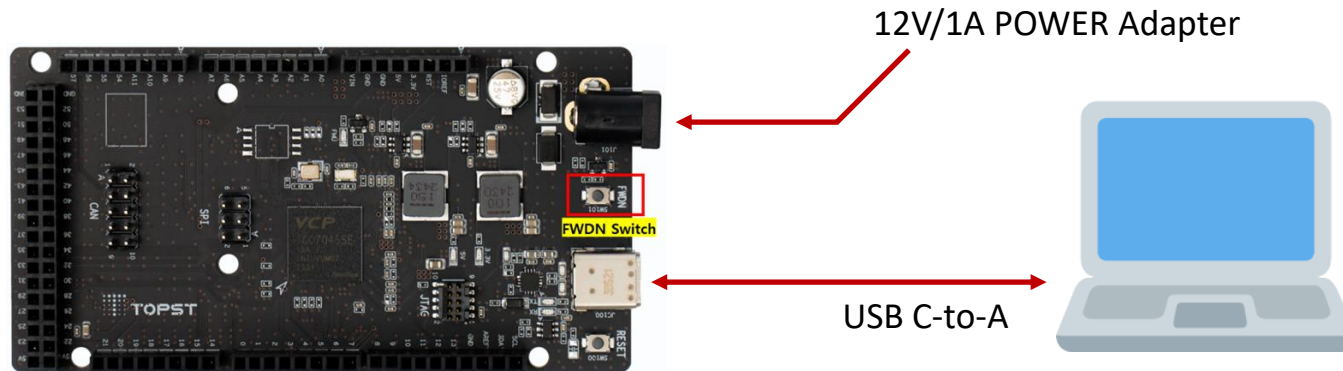


Figure 2.1 Block Diagram of Software Components in TCC70xx MCU BSP

- VCP-G에선 Realtime OS로 FreeRTOS & uCOS/III 지원
- 본 과정에선 FreeRTOS를 사용하여 GPIO 와 CAN 통신을 위한 예제만 사용

VCP-G 환경 설정



- **FWDN (FreeRTOS)**

- USB Type-C 케이블을 Host PC 및 VCP-G의 USB Type-C FWDN 포트에 연결
- FWDN 스위치를 누른 상태에서 전원 케이블을 VCP-G 보드에 연결 (**FWDN 모드 진입**)
 - VCP-G 의 전원 케이블은 **12V/1A 케이블만 사용**

VCP-G 환경 설정

선택 관리자: Windows PowerShell

```
PS C:\Windows\system32> usbipd list
Connected:
BUSID VID:PID DEVICE STATE
3-1 0bda:8153 Realtek USB GbE Family Controller Not shared
5-3 04e8:a057 Samsung USB C Earphone, USB 입력 장치 Not shared
5-7 27c6:6594 Goodix MOC Fingerprint Not shared
5-9 5986:1199 Integrated Camera, Integrated IR Camera, Camera DFU Device Not shared
5-10 8087:0036 인텔(R) 무선 Bluetooth(R) Not shared
5-13 0bda:8153 Realtek USB GbE Family Controller #2 Not shared
6-2 17ef:30b0 ThinkPad USB-C Dock Audio, USB 입력 장치 Not shared
6-3 17ef:30a9 Billboard Device, Vendor Interface Not shared
7-2 10c4:ea60 Silicon Labs CP210x USB to UART Bridge(COM7) Shared
7-3 067b:2303 Prolific USB-to-Serial Comm Port(COM18) Not shared
7-4 067b:2303 Prolific USB-to-Serial Comm Port(COM17) Not shared
9-2 046d:c52b Logitech USB Input Device, USB 입력 장치 Not shared
9-3 046d:c548 Logitech USB Input Device, USB 입력 장치 Not shared
```

```
ben@TC-E-20-0385 ~/$ sudo dmesg | grep tty
[ 88.186711] usb 1-1: cp210x converter now attached to ttyUSB0
ben@TC-E-20-0385 ~/$
```

• Connect Hardware to Host PC

- Run PowerShell and attach the VCP-G (**Windows환경에서** PowerShell 실행-관리자권한)
 - \$ usbipd list
 - \$ usbipd bind --busid <busid>
 - \$ usbipd attach --wsl --busid <busid>
- Connect USB Type-C Cable & Verify USB Connection (**WSL2[Ubuntu]에서 실행**)
 - \$ sudo apt-get install usbutils && lsusb
 - \$ sudo dmesg | grep tty

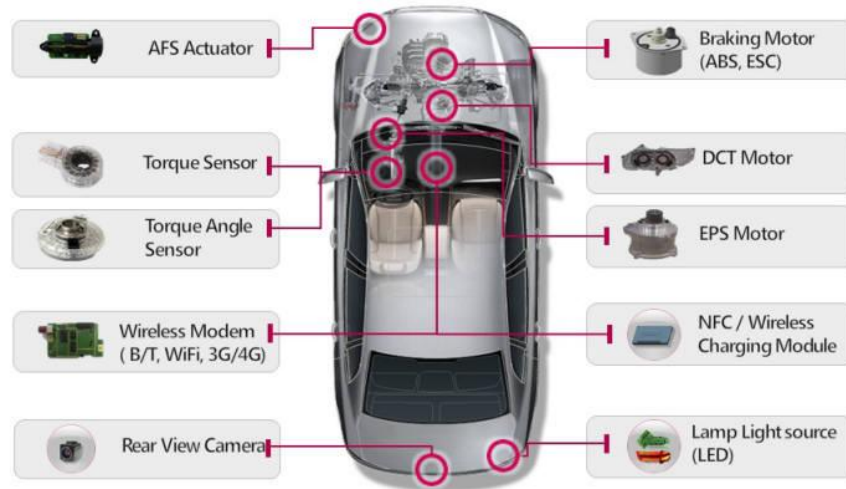
VCP-G 빌드 환경 세팅

- **FreeRTOS 소스코드 다운로드 (WSL에서 수행)**
 - FreeRTOS Repository edu 폴더로 클론
 - `$ cd`
 - `$ git clone https://github.com/topst-development/FreeRTOS-VCP.git edu`
 - edu 폴더로 이동
 - `$ cd edu`
 - edu/motrex_v2 브랜치로 체크아웃
 - `$ git checkout edu/motrex`
 - `$ ./easy-setup_vcp-g.sh`
 - “->” 키 누른 후 엔터 입력

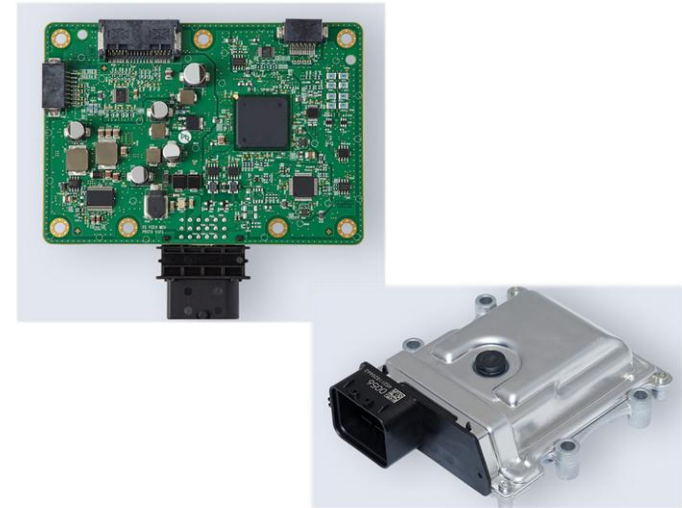
빌드 & FWDN

- build 진행
 - `$ cd ~/edu/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/edu/tools/fwdn_vcp/fwdn`
`--fwdn ~/edu/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/edu/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- 디버깅을 위해 아래 명령어 입력 후 board의 reset 버튼 누르기
 - `$ minicom -D /dev/ttyUSB0 -b 115200 -8`

차량 내 전장장치들과 제어기



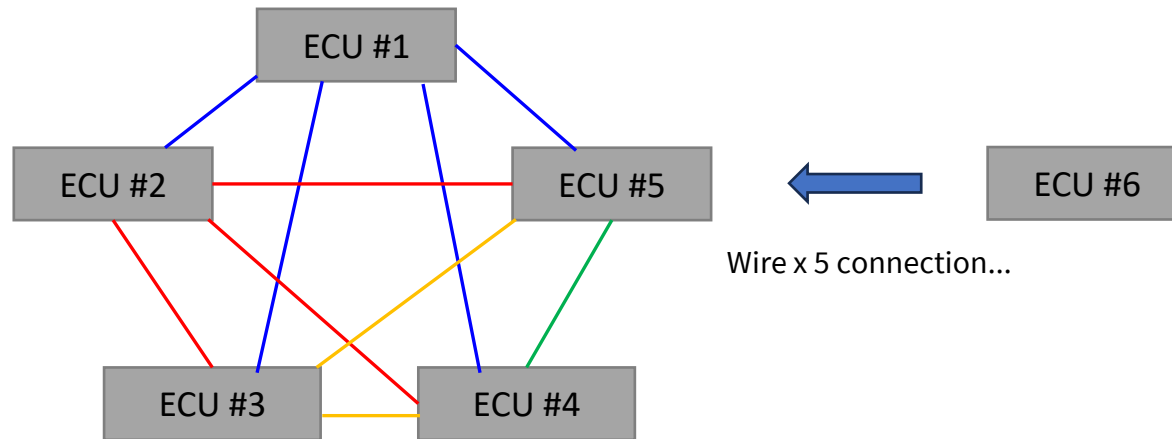
전장장치들의 예시



제어기의 예시

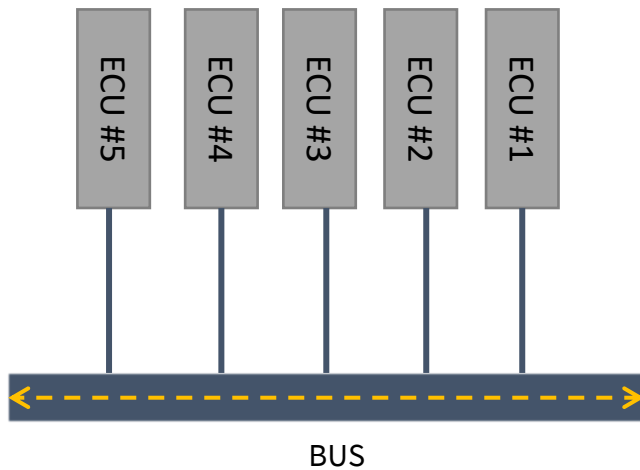
- 전자장치 : 전기&전자적으로 동작하는 차량 부품(ex. 액츄에이터, 모터, 센서 등..)
- 제어기(Controller) : 차량 내에 구성된 수많은 장치들을 제어할 수 있는 물리적인 하드웨어
- 기능이 고도화될 수록 전자장치의 수가 증가되며, 이를 제어하는 제어기의 역할이 중요함
- 각 장치들끼리 통신을 해야 하는 기술적 수요가 필연적임
- 제어기끼리 통신을 한다면 과연 어떻게 해야 할까.

ECU Wire Connection



- 만약 ECU의 통신라인이 위와 같은 구조라면 ??
- 제어기가 추가될 때마다 통신라인 추가
- 비용과 함께 차량 무게 증가
 - 생산성 & 안전성 부적합

ECU Wire Connection

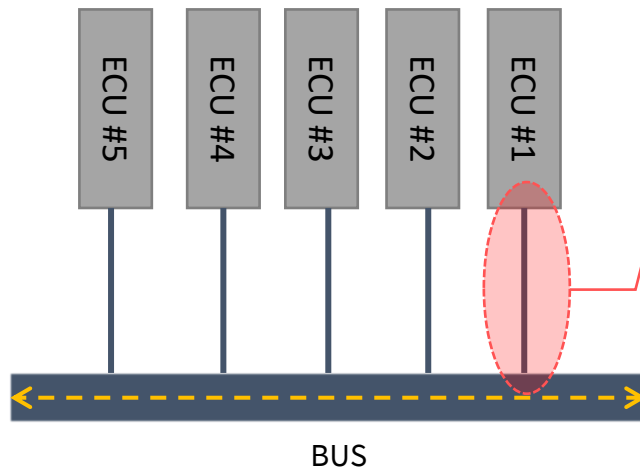


- CAN

- **C**ontroller **A**rea **N**etwork
- Broadcasting 방식의 network (src/dst 지정x)
- 제어기 간 네트워크 효율화를 위해 설계 및 개발(BOSCH)
- 현재까지 모든 OEM에서 차량 네트워크로 CAN 통신을 채택함

1983	Bosch社에서 차량용 네트워크 프로토콜 개발 프로젝트 착수
1986	CAN 프로토콜이 Society of Automotive Engineers (SAE)에 공식적으로 소개됨
1987	Intel社와 Philips社에 의해 첫 번째 CAN 칩이 생산됨
1991	Bosch, CAN 2.0 사양 발표
1992	Mercedes-Benz社에서 최초로 CAN 적용 차량 생산
1993	ISO 11898 (data link and high-speed physical layer) 사양 제정
1994	SAE J1939 사양 제정
1995	ISO 11898 사양 개정 (extended frame format)
2003	Data link 계층과 high-speed physical 계층 사양 분리 (ISO 11898-1 / -2)
2006	ISO 11898-3 (low-power, low-speed physical layer) 사양 제정
2007	ISO 11898-5 (low-power, high-speed physical layer) 사양 제정
2011	CAN FD 프로토콜 개발 착수
2013	ISO 11898-6 (physical layer with selective wake-up function) 사양 제정
2015	ISO 11898-1 (Classical CAN and CAN FD) 사양 발표

CAN Connector



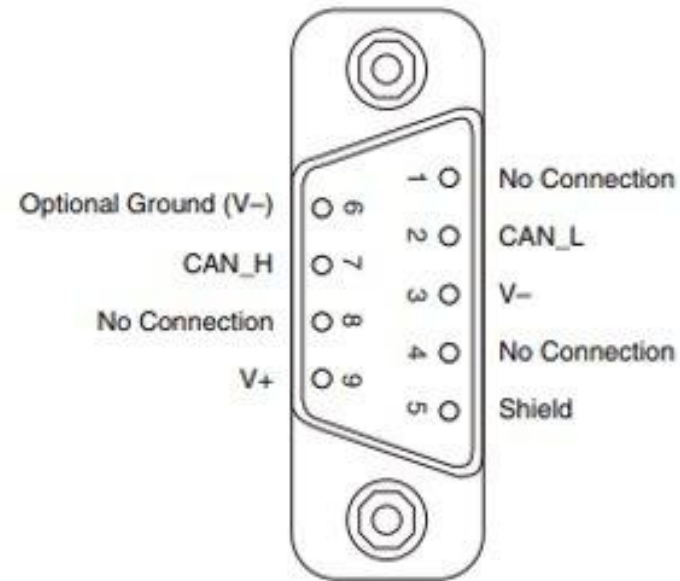
- 제어기끼리 BUS와 통신을 할 땐, 어떤 커넥터를 사용?
 - USB C type?
 - USB A type?
 - ...
 - 만약, 서로 다른 제어기 회사끼리 자신들이 만든 커넥터를 사용한다면, **커넥터의 호환성 문제**가 발생



CAN Connector



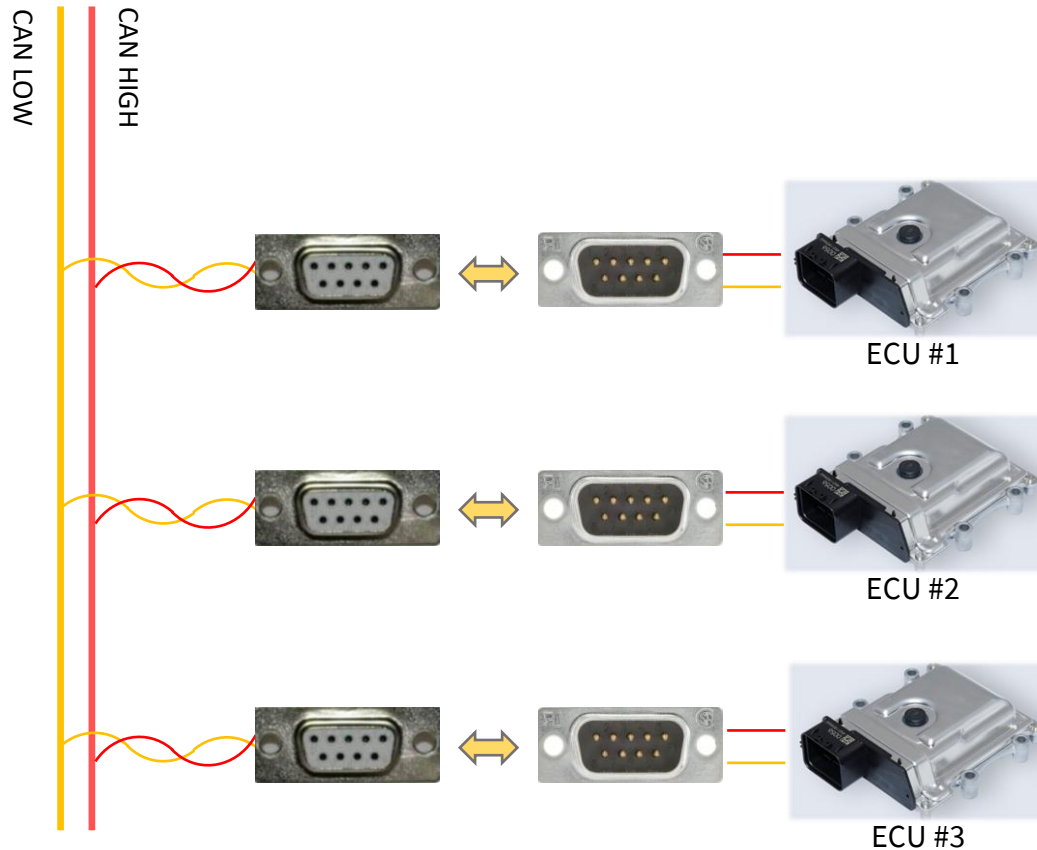
D-Sub 9pin Connector



D-Sub 9pin Connector 구조

- CAN 통신을 위한 포트로서 D-Sub 9 pin 커넥터 사용을 **약속**
- D-Sub 포트에 연결되는 connect hole도 약속
 - 2번 – CAN Low signal을 위한 wire
 - 7번 – CAN High signal을 위한 wire
- 대부분의 제어기 & 디버깅 장비는 D-sub 9 pin 포트 지원을 함

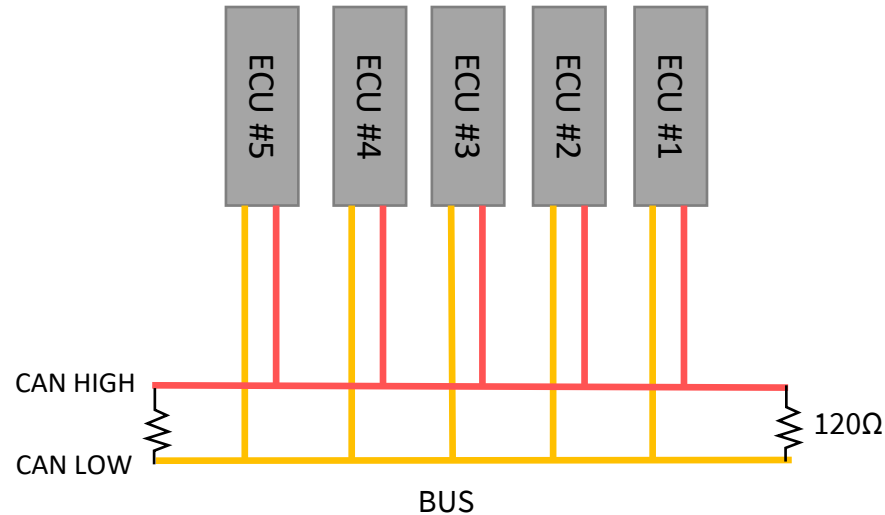
CAN Connector



CAN connection

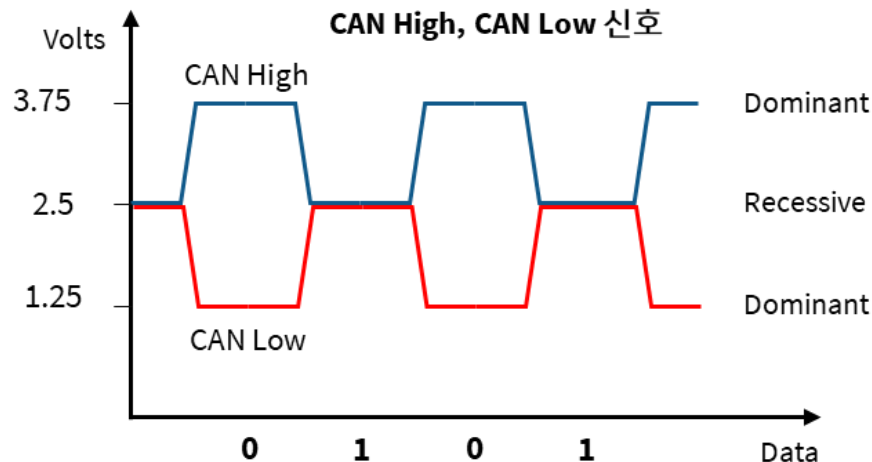
- 여러 개의 제어기끼리 통신한다면,
- CAN High, CAN Low 시그널의 버스 안에서,
- 통신에 참여하고 싶은 커넥터에 각각 연결하게 되면 CAN 통신 가능

CAN High & Low



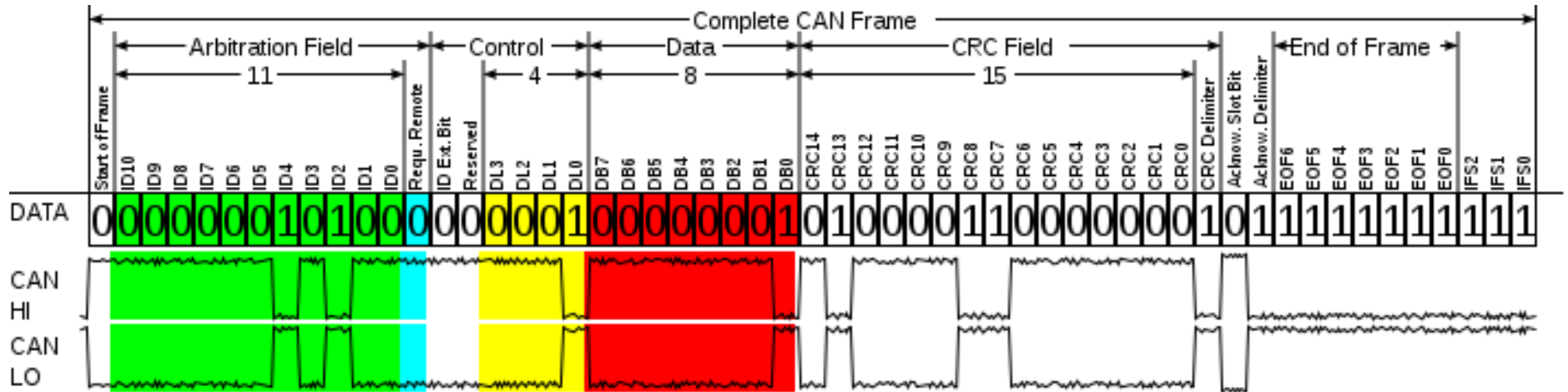
- 통신에서 메시지를 보낸다는 것은 전선에 어떤 전기적인 신호를 발생시켜 전달함을 의미
- Computer 체계에서 모든 정보는 binary 정보(0과 1)로 표현된다.
- CAN 통신은 특별히 HIGH와 LOW 두 개의 라인으로 0과 1을 표현함
- 그렇다면 과연 어떻게 0과 1로 표현할 수 있을까

CAN High & Low



- [RULE] CAN High에 걸리는 전압과 Low에 걸리는 전압의 차를 이용해서 0,1 을 표현한다.
 - CAN High – CAN Low = 0.9 ~ 5 V → 0 (Dominant)
 - CAN High – CAN Low = -0.1 ~ 0.5 V → 1 (Recessive)
- CAN High 와 Low 파형은 서로 완벽하게 대칭하는 특성을 보임
- Dominant (Data 0) 가 Recessive (Data 1)보다 우선 순위가 높음 (중요)
 - 여러 대수의 제어기가 하나의 버스로 연결이 되어 있다 가정
 - 모든 제어기가 동시에 신호를 보내는데, 우선순위를 어떻게 처리하는지에 대한 가이드가 정의되어야 함
 - 결과적으로 버스에는 Dominant가 우선순위가 높기 때문에 버스 전체에는 Dominant에 해당되는 전압이 출력 된다.

Overview of CAN Message Format



- CAN 에서 모든 메시지 전송은 브로드캐스트 방식으로 이루어짐
- 송신되는 메시지 하나는 위와 같이 구성되어 있음
- CAN 에서는 Source Address, Destination Address가 없음
- 따라서, CAN message 에서는 ID 영역이 존재
 - Arbitration Field 중 ID에 해당하는 비트가 총 11개 존재

CAN DB

Message ID	Message Name	Data Length (byte)	Transmitter	Receiver	Period
0x123	Steering_status	4	Steering Controller	Module A controller	100ms
0x145	Engine_status	3	Engine Controller	Module B Controller	200ms
0x167	Battery_status	4	BMS	Module C Controller	100ms

CAN DB Example

- CAN 메시지 송신자, 수신자를 구분하기 위해 메시지 ID를 사용함
- OEM에서 ID별로 메시지 이름을 정하고, 해당 메시지의 송수신 제어기가 어디인지 정의하며 이런 유형의 모든 내용이 정리된 데이터를 CAN DB라고도 부른다.
- 보통 ID는 16진수로 표현됨
- CAN 메시지는 네트워크 안에서 각각 고유한 ID를 보유하는 것이 원칙이다.
 - 다만, 차량 내부에는 여러 CAN 네트워크 영역이 존재하며,
 - CAN DB도 서로 별도로 존재하여 네트워크 영역 간 서로 관련 없는 독립적인 정보이다.

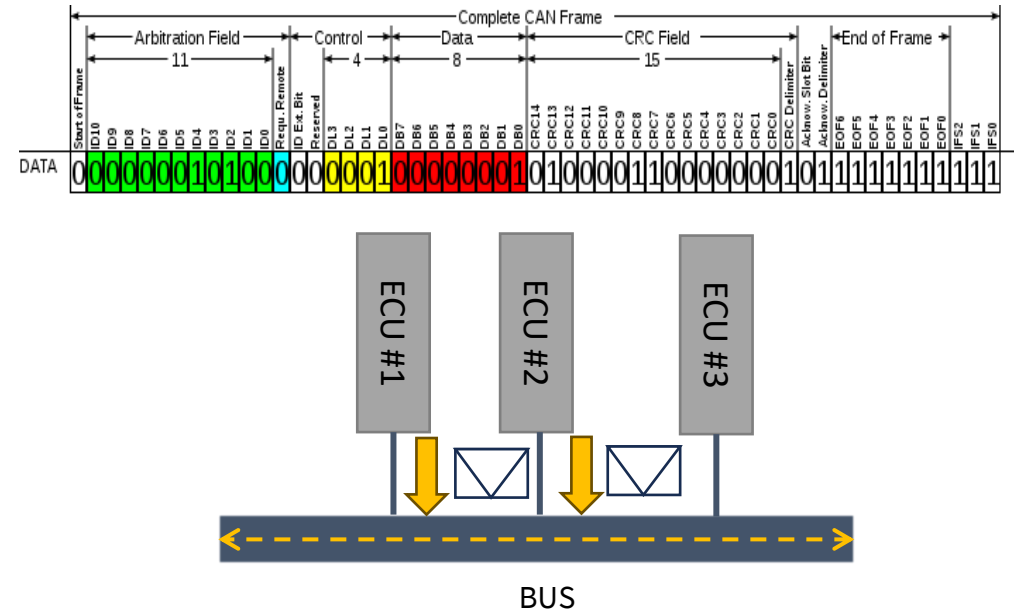
Message's Priority

Message ID	Message Name	Transmitter	Receiver
0x123	Steering_status	Steering Controller	xxx
0x145	Engine_status	Engine Controller	xxx
0x167	Battery_status	BMS	xxx

0x123 = 00100100011

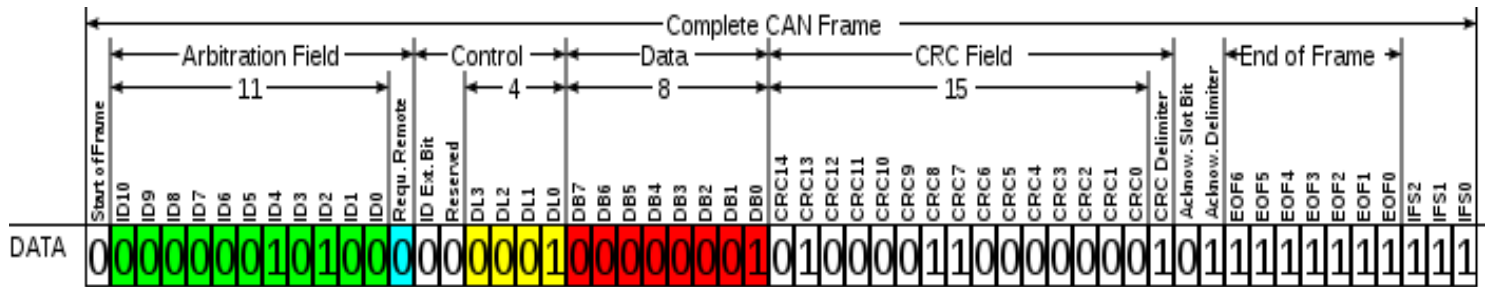
0x145 = 00101000101

0x167 = 00101100111



- CAN 통신에서는 특정한 순간에 버스 전체에 오직 하나의 제어기가 하나의 메시지만 수신할 수 있음
- ECU1과 ECU2에서 동시에 메시지를 보내려고 한다면?
 - 한번에 여러개의 메시지를 보낼 수 없는 CAN 통신 특성상, 메시지의 우선순위를 따진다
 - Data 0 = Dominant , Data 1 = Recessive
 - Priority : Dominant > Recessive
 - ID값이 작을수록 우선순위가 높다.
 - 각각의 제어기들은 현재 네트워크에 다른 제어기가 메시지를 보내고 있는지 아닌지를

계속 체크

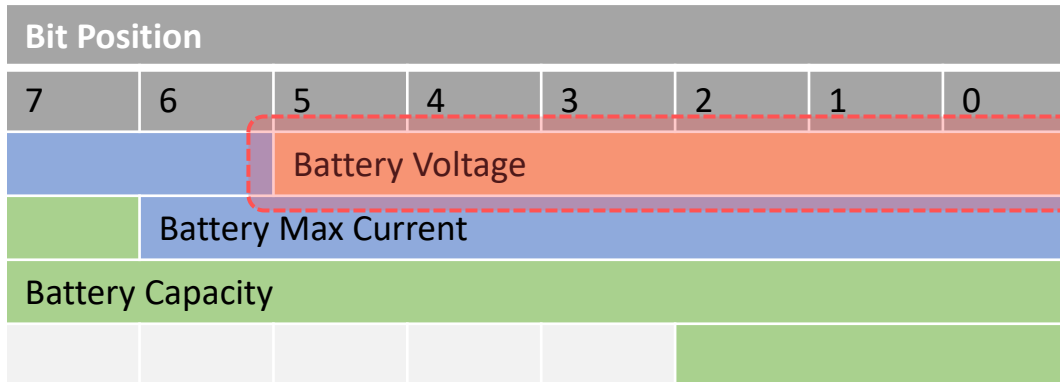


- 시그널 : 메시지 안에 구체적으로 담겨 있는 무언가의 정보
- Data Length Code (DLC)
 - CAN 메시지는 데이터 영역의 사이즈가 고정이기 때문에 메시지의 데이터 영역에 데이터가 몇 바이트 담겨 있는지를 수신자에게 정보를 제공해야함
 - DLC는 CAN 메시지의 데이터 영역의 길이가 몇 바이트인지를 알려준다.
- CRC Field
 - 전송 메시지의 오류를 검사
- ACK Field
 - 메시지 받은 ECU에서 올바르게 수신했다고 체크
- EOF
 - 프레임의 종료를 나타냄

Signal & Data length Code

Message ID	Message Name	Data Length (byte)	Transmitter	Receiver	Period
0x123	Steering_status	4	Steering Controller	xx,xx,xx	100ms
0x200	Engine_status	3	Engine Controller	xx,xx,xx	200ms
0x300	Battery_status	4	BMS	xx,xx,xx	100ms

CAN DB Example



Message Signal Example

Message Signal Example

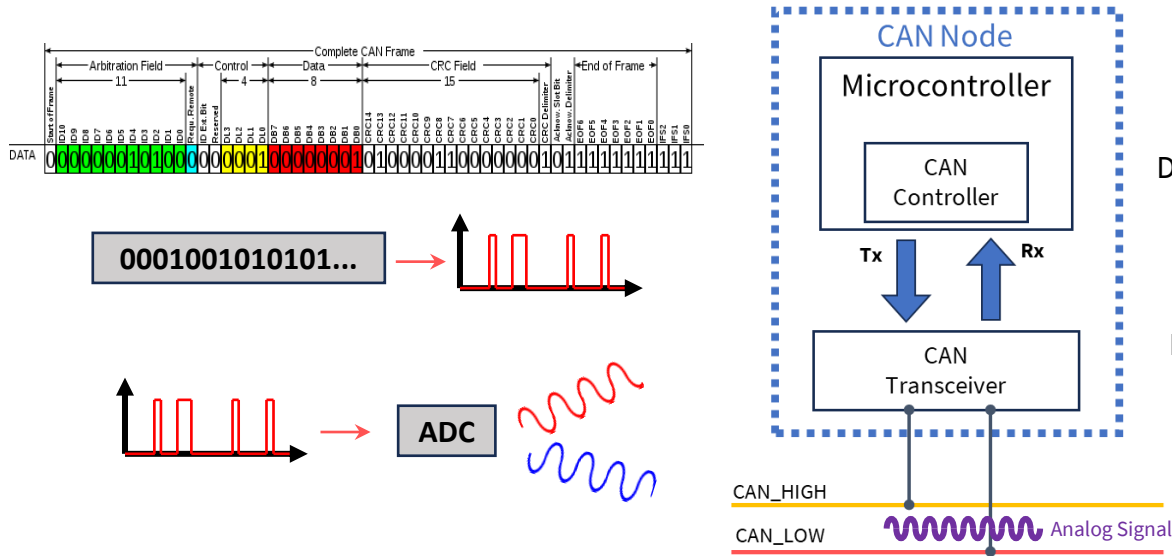
Message ID	Signal List
0x300	Battery Voltage
	Battery Max Current
	Battery Capacity

Battery Voltage Signal Detail Information

- Start bit : 0
- Length : 6
- Unit : V
- Offset / Factor : xxx/xxx

- ID가 0x300 인 메시지는
 - Battery 정보를 담는 CAN 메시지
 - Battery Voltage, Max current, Capacity 시그널을 포함
 - DLC : 4Byte

CAN Message 전송 흐름



Data Link Layer
ISO 11898-1

Physical Layer
ISO 11898-2/
11898-3

ISO 11898-1: CAN 데이터 링크 계층을 정의한다. CAN 프로토콜에서 데이터의 전송 및 오류 처리 기능 뿐만 아니라 표준 CAN 메시지의 구조와 프레임의 형식을 규정한다.

ISO 11898-2: 고속 CAN(CAN High-Speed)의 물리 계층을 정의한다. 차량 내 통신에서 주로 사용되며, 데이터 전송 속도는 최대 1 Mbps이다.

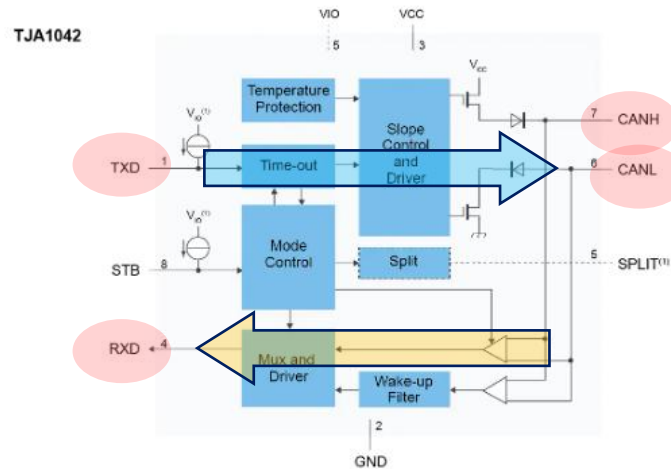
ISO 11898-3: 저속 CAN(CAN Low-Speed)의 물리 계층을 정의한다. 주로 차량의 간단한 네트워크 및 저속 통신에 사용되며 Fault Tolerant(내고장성, 결함 허용) 특징을 가지고 있다.

- CAN 통신과 연관된 동작을 하는 장치 : CAN controller
 - CAN controller 는 MCU 내부에 마련되어 있음
 - CAN 통신의 프로토콜을 따라서 실제로 CAN 통신을 할 수 있도록 지원
- CAN 컨트롤러에서 링크 계층 표준을 만족하며, 데이터의 전송, 수신 및 프로토콜을 담당한다.
- CAN Transceiver(**T**ransmitter + **R**eceiver)
 - 디지털 신호,아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
 - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
 - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

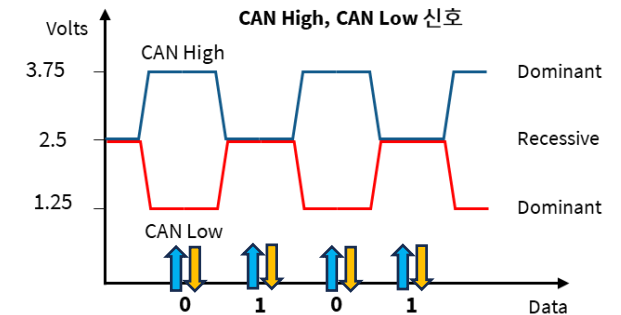
CAN Transceiver



CAN Transceiver 칩 (NXP JTA1042)

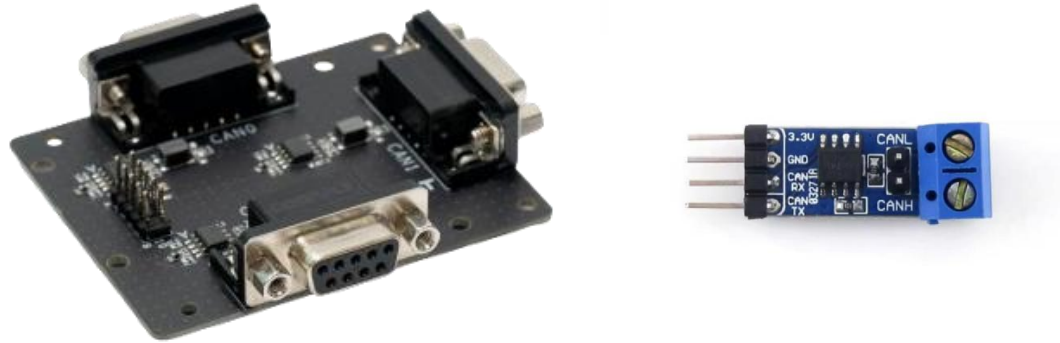


NXP JTA1042 회로도



- MCU는 3.75V, 2.5V, 1.25V 의 전압을 출력할 수도 없고, 반대로 전압을 읽어서 신호가 0인지 1인지 해석할 수 없음
- 따라서 CAN Transceiver(**Transmitter** + **Receiver**)는 CAN 통신에서 필수적으로 요구되는 장치
 - 디지털 신호,아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
 - 전위차를 이용해 0,1 신호로 변환
 - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
 - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

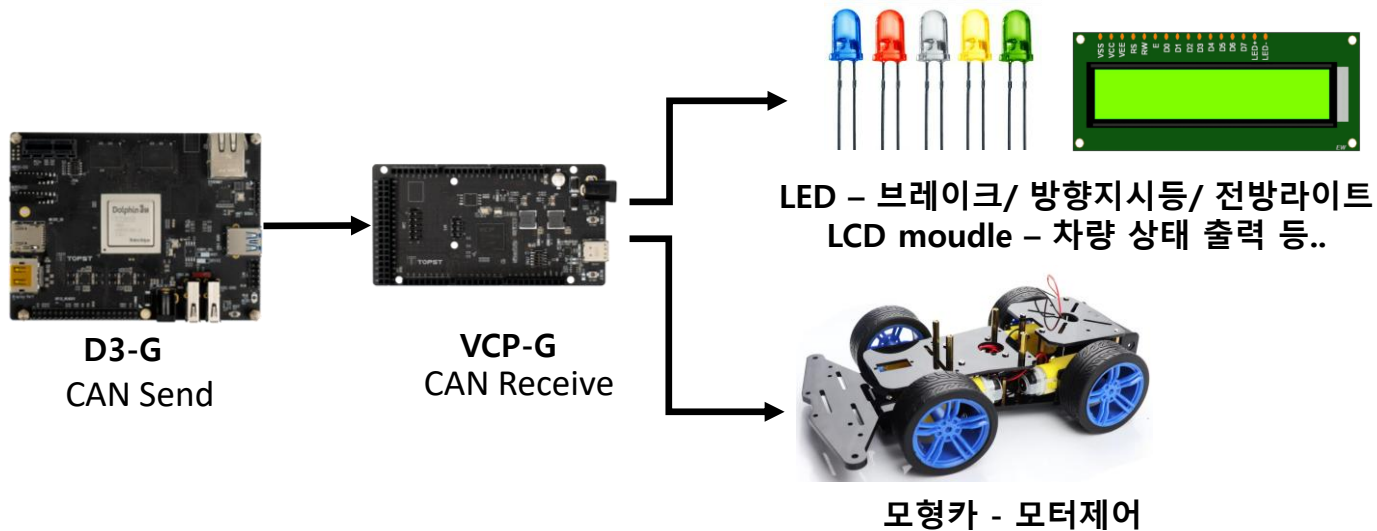
CAN Transceiver



CAN Transceiver 장치들

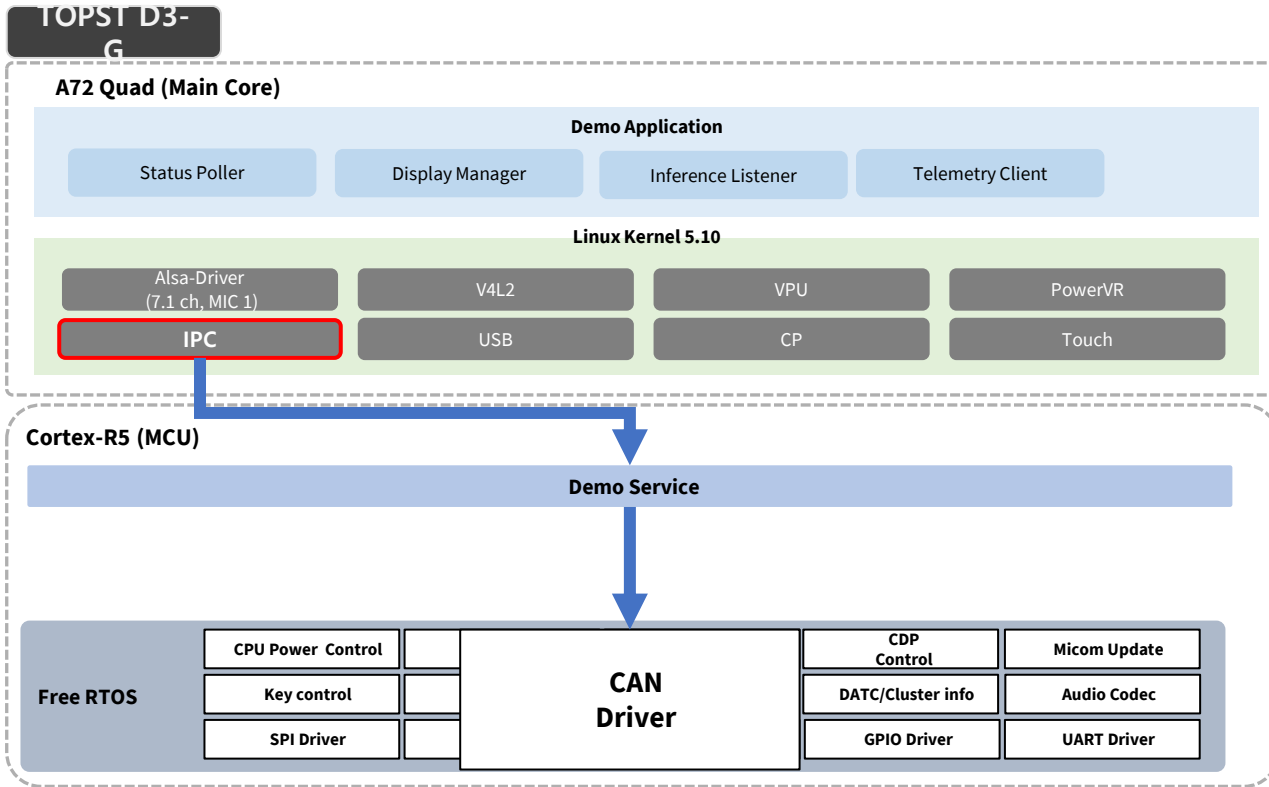
- 대부분 Transceiver 는 D-sub 9pin 포트로 지원 혹은,
- 간단하게 High, Low 하나의 채널만 사용할 수 있게 pin header 로 지원

CAN TASK 실습 구성도



- D3-G에서 CAN 메시지 송신
- VCP-G 에서 CAN 메시지 수신
- 수신된 결과에 따라 장치 제어

D3-G CAN 통신 구조



*IPC : Inter Processor Communication

1. D3-G to MCU IPC를 이용해 상태 전달
 - CAN Message와 유사한 별도의 User-defined protocol을 사용
2. MCU에서 user-defined protocol을 수신하고 해석
3. Micom에서 CAN 통신 포트 설정 (init, open 등)
4. 전송할 항목을 CAN 통신 프로토콜로 전송

이번 실습은 CAN 통신 실습을 위해 TOPST D3-G 보드를 사용하며 코어당 수행하는 기능을 **통합하여 데모**

- A72 core : IPC Task 수행
- R5 core : CAN Controller 수행

D3-G CAN 실습 사전 조건 - IPC TASK 이해하기



D3-G chip : Dolphin3

- IPC : Inter Processor Communication 의 약자로, **CPU와 MCU 간의 통신을 지원하기 위한 통신 구조**

Q) CAN 드라이버를 CPU에서 직접 안 쓰나요?

➔ A72는 기존 IVI를 담당하는 코어, A72가 CAN 까지 관리한다면, 코어가 예기치 못한 오류로 죽을때, 차량 제어 영역이 매우 critical해짐.

Q) 그래서 CPU가 CAN을 사용하기 위해선?

➔ CPU > IPC > MCU > CAN 드라이버 > 차량 네트워크

쉽게 설명하여, IPC 는 CPU가 MCU에게 "이 메시지를 좀 보내줘" 라고 전달하는 우체부 역할

D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

- IPC_Example.py의 전체 동작 구조는 아래와 같음

1. A72 쪽 사용자 입력 (IPC_Example.py)

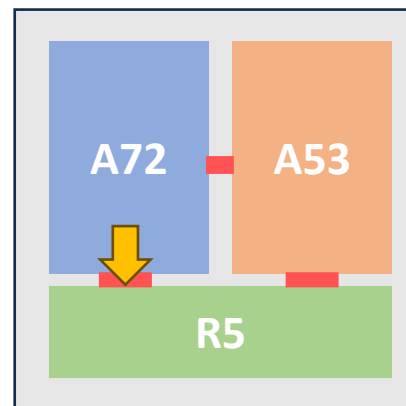
- argparse로 모드와 데이터를 입력 받음 (snd /rev / sndrcv)
- snd : send
- rev : receive
- sndrcv : send and receive

2. 패킷 구성 및 송신 요청

- IPC_SendPacketWithIPCHheader() 호출
- 데이터 -> 헤더 + 명령 + CRC 구조로 패킹
- /dev/tcc_ipc_micom 파일로 전송

3. R5 쪽 수신 및 응답

- R5 가 패킷을 수신 -> CAN 드라이버로 전달 (rev 모드에서는 응답 패킷 수신)



D3-G chip : Dolphin3

D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

• IPC_Example.py 내 main()

```
def main():
    parser = argparse.ArgumentParser(description="IPC Sender/Receiver")
    parser.add_argument("mode", choices=["snd", "rev", "sndrecv"], help="'snd': send, 'rev': receive, 'sndrecv': send then receive once")
    parser.add_argument("--file_path", default="sndfile", help="File path for IPC communication") # sndfile = open("/dev/tcc_ipc_micom", 'wb')
    parser.add_argument("--channel", type=int, default=1, help="can channel bit flag: b111")
    parser.add_argument("--txOnly", type=int, default=0, help="tx only channel bit flag: b111 (dec: 1, 2, 4) ")
    parser.add_argument("--canID", type=parse_hex16, default=0, help="CAN ID max 16-bit: e.g. 123 or 0x1234")
    parser.add_argument("--sndDataHex", type=str, help="Value for sndData as a hex string, e.g., '12345678' (default: string)")
    parser.add_argument("--sndData", type=str, default="12345678", help="Default data if no sndDataHex provided")

    ...

    sndDataLength = len(sndData)
    channel_str = ", ".join(parse_channels(channel_bitmask))
    txonly_str = ", ".join(parse_channels(tx_only_channel_bitmask))
    print(f"channel_bitmask: 0x{channel_bitmask:02X} (channels: {channel_str or 'None'})")
    print(f"tx_only_channel_bitmask: 0x{tx_only_channel_bitmask:02X} (channels: {txonly_str or 'None'})")
    print(f"canID: 0x{canID:04X}")
    print(f"sndData: {sndData}")
    print(f"sndDataLength: {sndDataLength}")
    ret = IPC_SendPacketWithIPCHheader(args.file_path, channel_bitmask, tx_only_channel_bitmask, canID, sndData)
```

- --file_path 는 /dev/tcc_ipc_micom 을 기본으로 사용
- IPC_SendPacketwithIPCHheader 함수를 통해, canID, sndData 전송에 중점
- 전송모드 snd, recv, sndrecv 명시 가능하나,
- --channel : CAN 통신 채널을 의미하며 1, 2, 4는 각각 0번, 1번, 2번 채널을 의미
- --canID : CAN 노드의 Identifier 를 의미
- --sndDataHex : 전송하고자 하는 전체 data를 의미하며 HEX 값으로 2자리씩 붙여서 명시한다 (ex : 0x123456 를 전송할때 '123456'로 명시)

D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

- IPC_SendPacketWithIPCHeader()의 구조는 아래와 같음
- function definition

IPC_SendPacketWithIPCHeader(sndFile, channel_bitmask, tx_only_channel_bitmask, canID, pucData)

- function flow

1. 빈 패킷 생성 (array)
2. 헤더 작성 (SYNC, START1, START2)
3. 길이 설정 (LENGTH)
4. CM3 추가 (Channel, txOnly, CAN ID)
5. 데이터 삽입(pucData)
6. CRC 계산 후 붙이기
7. bytes로 변환
8. /dev/tcc_ipc_micom에 write()

```
packet_send = array.array('B', [0] * IPC_MAX_PACKET_SIZE)
...
cnt = 0
for cnt in range(pucData_len):
    packet_send[IPC_PACKET_PREPARE_SIZE + uiCmd3_size + cnt] = pucData[cnt]
...
packet_bytes = bytes(packet_send[:total_size])
...
while True:
    try:
        sndFile.write(packet_bytes)
        now = time.time() * 1000
        sndFile.flush() #
    ...
```

A72 IPC 세팅 & 실행

1. 1일차에 FWDN 한 ubuntu d3-g를 mobaXTerm으로 연결한다. (id : TOPST / passwd : TOPST)
2. 아래와 같은 커맨드를 입력해서, tcc_ipc_micom 가 설정되어 있는지 확인한다.

```
topst@TOPST:~$ ls -al /dev/tcc_ipc*
crw----- 1 root root 244, 0 Jun  4 14:17 /dev/tcc_ipc_ap
crw----- 1 root root 245, 0 Jun  4 14:17 /dev/tcc_ipc_micom
topst@TOPST:~$
```

3. 아래와 같이 실습용 확인 (D3-G 보드)

```
$ cd motrex_education
```

```
$ ls
```

- IPC_Example.py 사용예정
 - IPC_Example.py - MCU로 메시지 송수신하는 IPC 코드 예제
 - IPC_Library.py - MCU로 메시지 송수신하는 IPC 라이브러리

```
├─ IPC_Example.py
└─ Library
   └─ IPC_Library.py
```

A72 IPC 세팅 & 실행

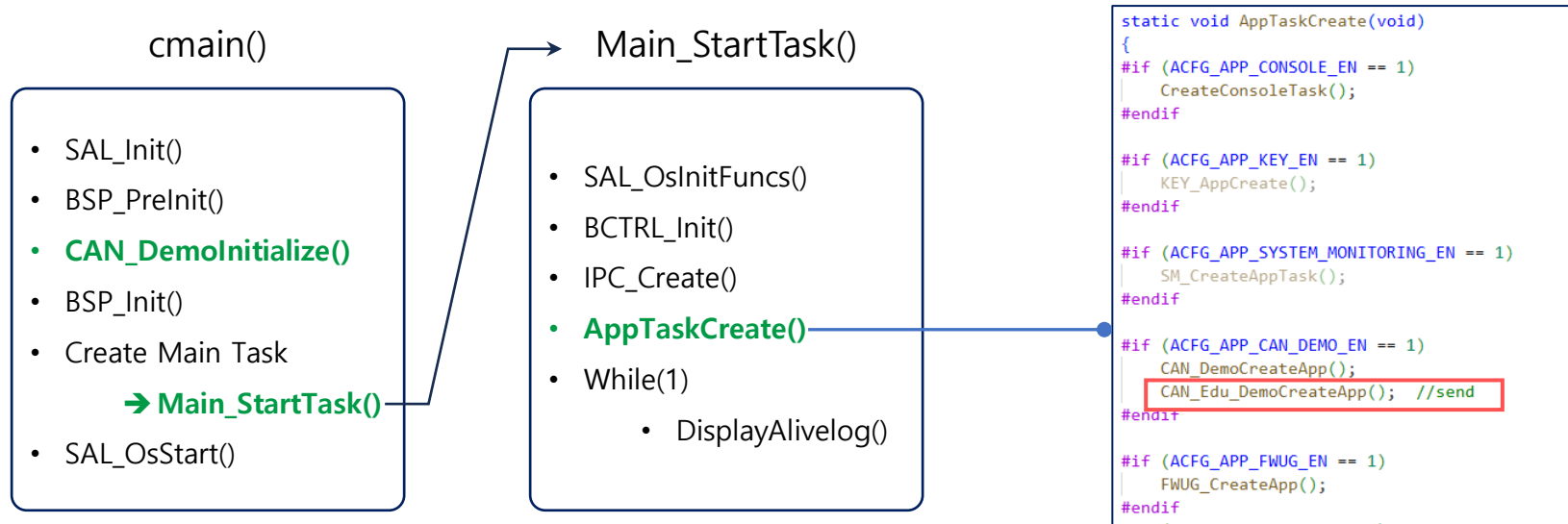
• IPC 전송 예제 실행

- 시리얼 통신으로 연결된 Putty 로그를 통해 A72에서 전송한 메시지를 아래 커맨드로 확인할 수 있다.
- `$ sudo python3 ./IPC_Example.py snd --sndData 1122`

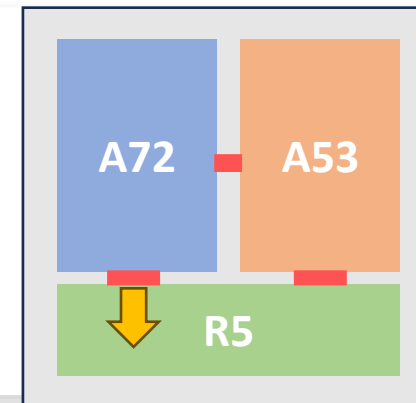
```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --sndData 1122
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0000
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0000
sndData: b'1122'
sndDataHex: 31313232
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

- CAN Channel , ID 는 default 값 사용
- Data 1122를 송신
- 출력되는 결과 : byte 1122 정상 출력완료
- 송신 데이터 길이 : 4 정상 출력완료

D3-G CAN 실습 사전 조건 – R5 CAN Task 이해하기



- D3-G R5 내에는 FreeRTOS 세팅
- 실질적으로 CAN TASK가 이루어지는 코어
- CAN_Edu_DemoCreateApp() 실행



R5 core : CAN Task 이해하기

```

static void CAN_Edu_DemoSend
(
    uint8                                ucCh
)
{
    CANMessage_t    canTxMsg;

    ...

    CANDemoData_t*  canData = &gDemoData[ ucCh ];

    ...

    // CAN 프레임 속성 결정
    canTxMsg.mBufferType    = CAN_TX_BUFFER_TYPE_QUEUE;
    canTxMsg.mExtendedId = (canData->value.id > CAN_DEMO_ID_MAX) ? TRUE : FALSE;
    canTxMsg.mId = canData->value.id;
    canTxMsg.mFDFormat = gNowMetaInfo.isFD;
    canTxMsg.mDataLength = canData->value.length;
    SAL_MemCopy( canTxMsg.mData, canData->value.data, canTxMsg.mDataLength ); //CAN 데이터 구성

    CANErrorType_t ret = CAN_SendMessage( ucCh, &canTxMsg, &canTxBufferIndex ); // CAN 메시지 전송 요청

    ...

}

```

```

typedef struct CANMessage
{
    CANMessageBufferType_t    mBufferType;
    uint8                     mBufferIndex;
    uint8                     mErrorStateIndicator;
    uint8                     mExtendedId;
    uint8                     mRemoteTransmitRequest;
    uint32                    mId;
    uint8                     mFDFormat;
    uint8                     mBitRateSwitching;
    uint8                     mMessageMarker;
    uint8                     mEventFIFOControl;
    uint8                     mDataLength; //byte unit
    uint8                     mData[CAN_DATA_LENGTH_SIZE];
} CANMessage_t;

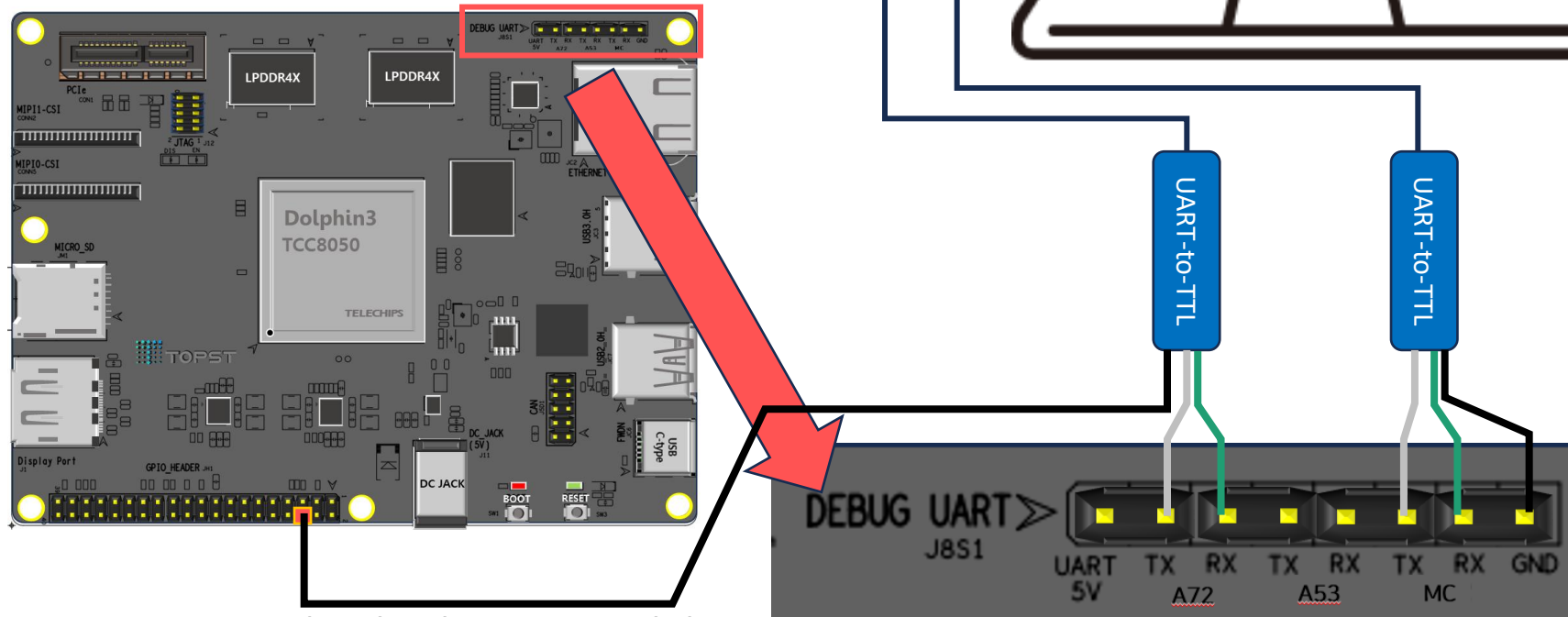
```

//CAN 송신 데이터 참조

- CAN 프레임을 구성하고 전송하는 핵심 송신 루틴

D3-G 보드 디버깅 환경 세팅

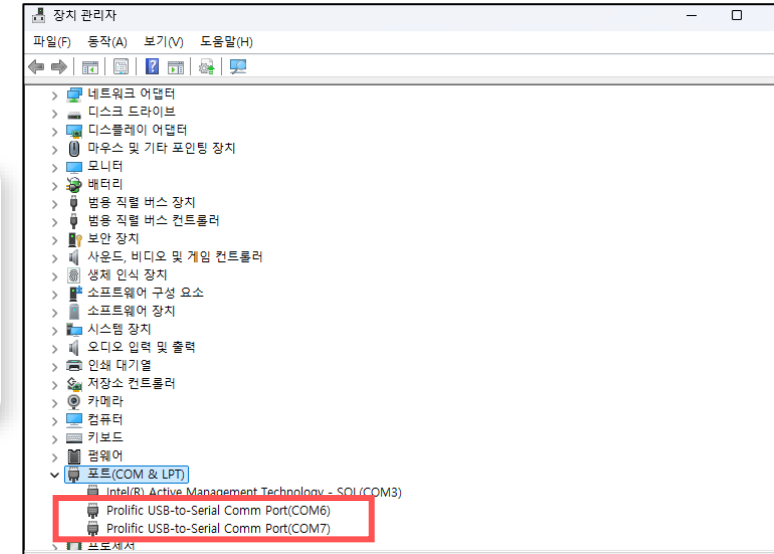
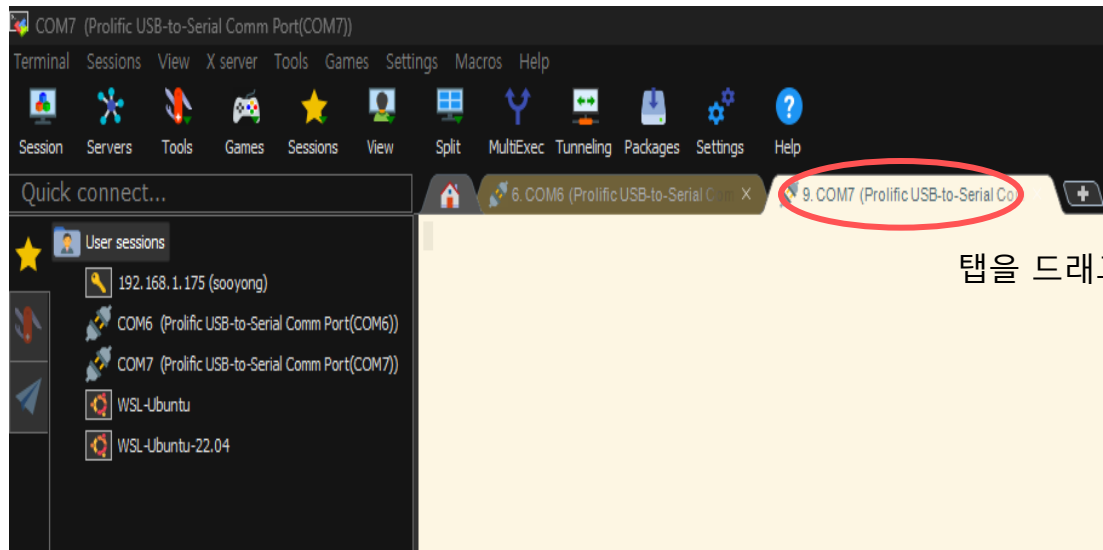
UART-to-TTL serial 케이블 2대를 동시에 사용하여
터미널 프로그램으로 각각 A72 / R5 코어 통신



반드시 6번Pin - GND 연결
연결 시, 점퍼선 활용

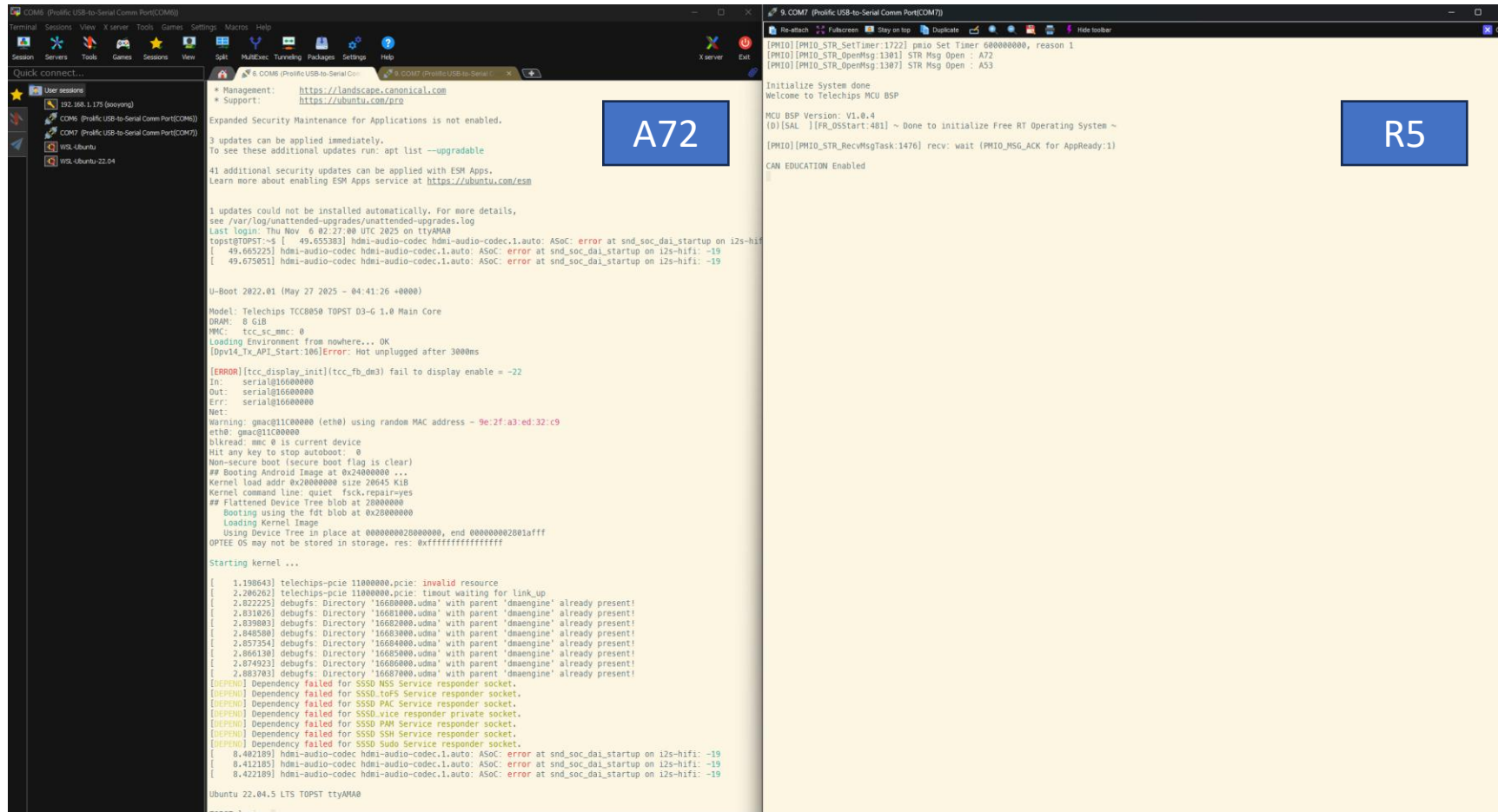
장치 관리자에서 확인

- COM 6 – A72
- COM 7 – R5



탭을 드래그해서 다른 화면에 새로 놓을 수 있음

D3-G 보드 디버깅 환경 세팅



D3-G CAN 통합 데모 시나리오

데모 순서는 아래와 같음

1. R5 부팅후, 아래 커맨드 입력

➤ log on

2. A72 에서 IPC 파이썬 스크립트 실행

```
$ cd motrex_education
```

```
$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
```

```
ch 0 : 1 (001)  001 > 0x001 [hex]          00112233 > 0x00112233
```

```
ch 1 : 2 (010)
```

```
ch 2 : 4 (100)
```

A72 화면

```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0001
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0001
sndData: b'\x00\x11"3'
sndDataHex: 00112233
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

D3-G CAN 통합 데모 시나리오

A72 화면

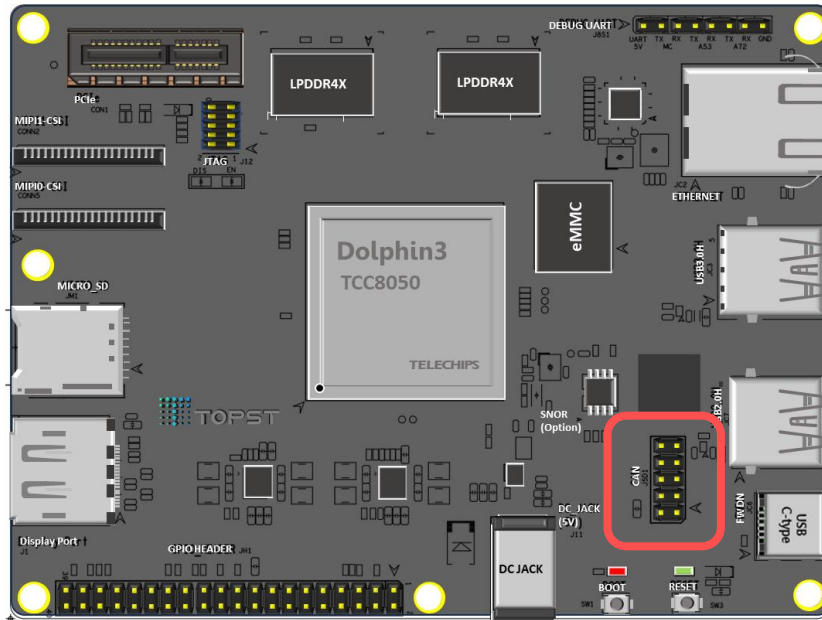
```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0001
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0001
sndData: b'\x00\x11"3'
sndDataHex: 00112233
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

R5 화면

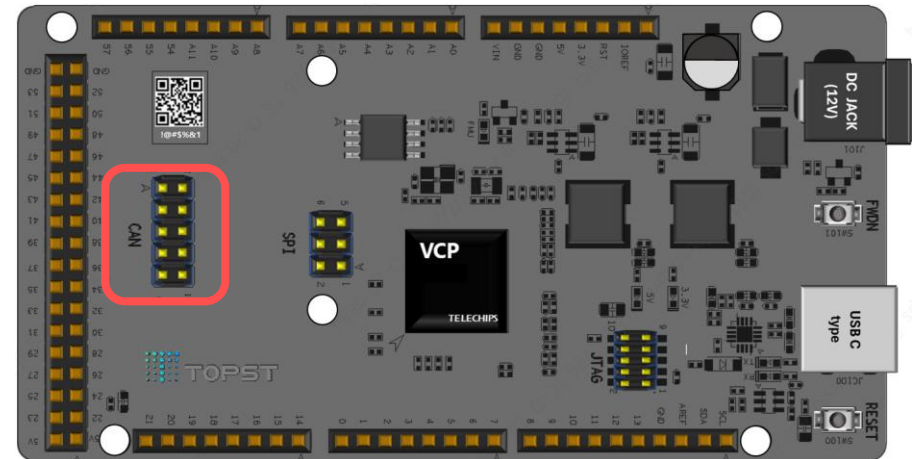
```
(D) [CAN ] [CAN_Edu_DemoSend:1353] *****
(D) [CAN ] [CAN_Edu_DemoSend:1354] [ID] : 0x1, [DATA SIZE] : 4, [DATA] :
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x00
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x11
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x22
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x33
(D) [CAN ] [CAN_Edu_DemoSend:1363]
(D) [CAN ] [CAN_Edu_DemoSend:1364] *****
```

Support to CAN pin

- Q) TOPST 보드는 어떤 형태로 CAN을 지원하는지
- TOPST D3-G CAN 지원은 3ch까지 가능 – 10개의 PIN 형태로 지원
- TOPST VCP-G CAN 지원은 3ch까지 가능 – 10개의 PIN 형태로 지원



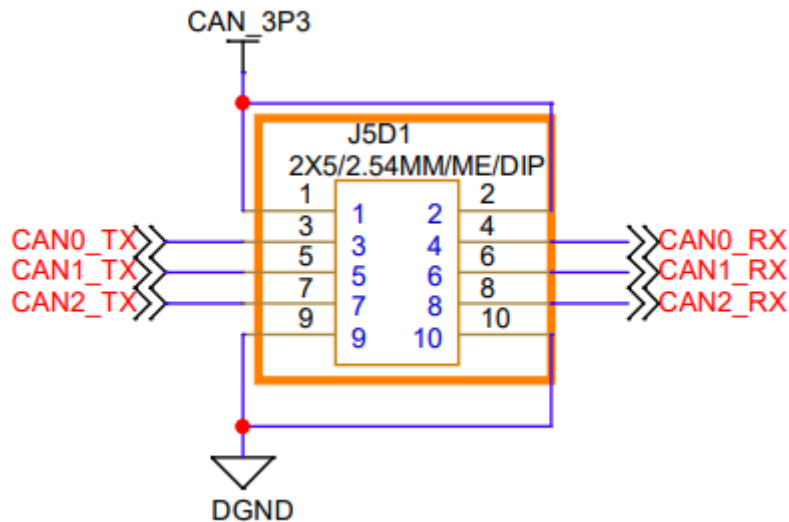
D3-G



VCP-G

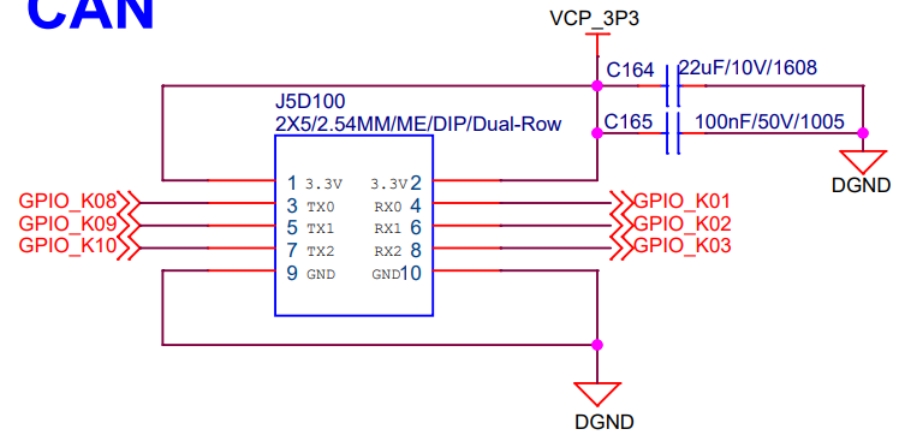
Support to CAN pin

- Q) TOPST 보드는 어떤 형태로 CAN을 지원하는지
- TOPST D3-G CAN 지원은 3ch까지 가능 – 10개의 PIN 형태로 지원
- TOPST VCP-G CAN 지원은 3ch까지 가능 – 10개의 PIN 형태로 지원
- 핀의 형태는 두 보드 동일



D3-G CAN Port

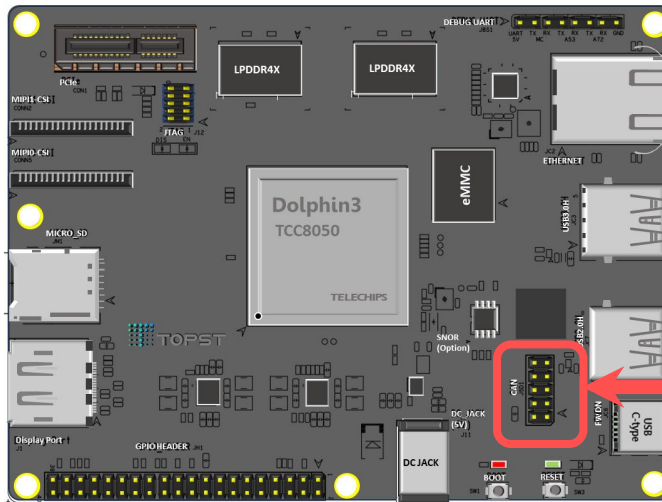
CAN



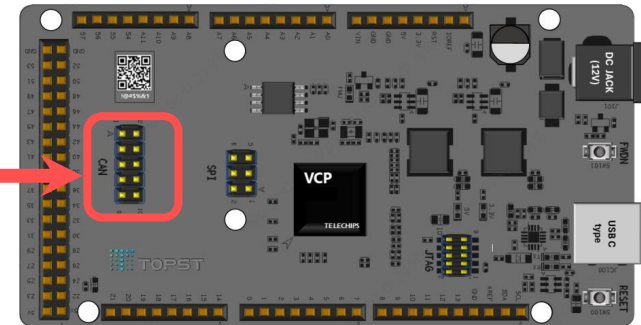
VCP-G CAN Port

Support to CAN pin

- Q) 물리적으로 서로 연결하기만 한다면 CAN통신이 되나요?
 - CAN 버스는 차동 신호(CAN High, CAN Low)로 통신하기 때문에 단순히 TX, RX 방식의 통신을 할 수 없음
- D3-G와 VCP-G의 CAN 통신을 위한 CAN Transceiver 장비를 사용
- 본 과정에서 SN65HVD230 VP230 CAN 모듈 사용 예정



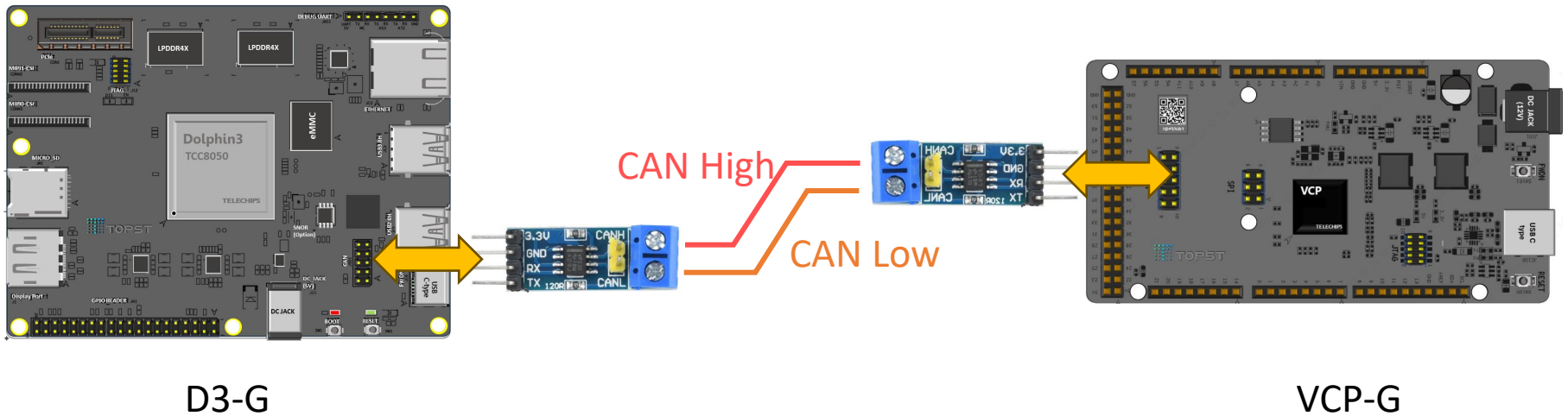
D3-G



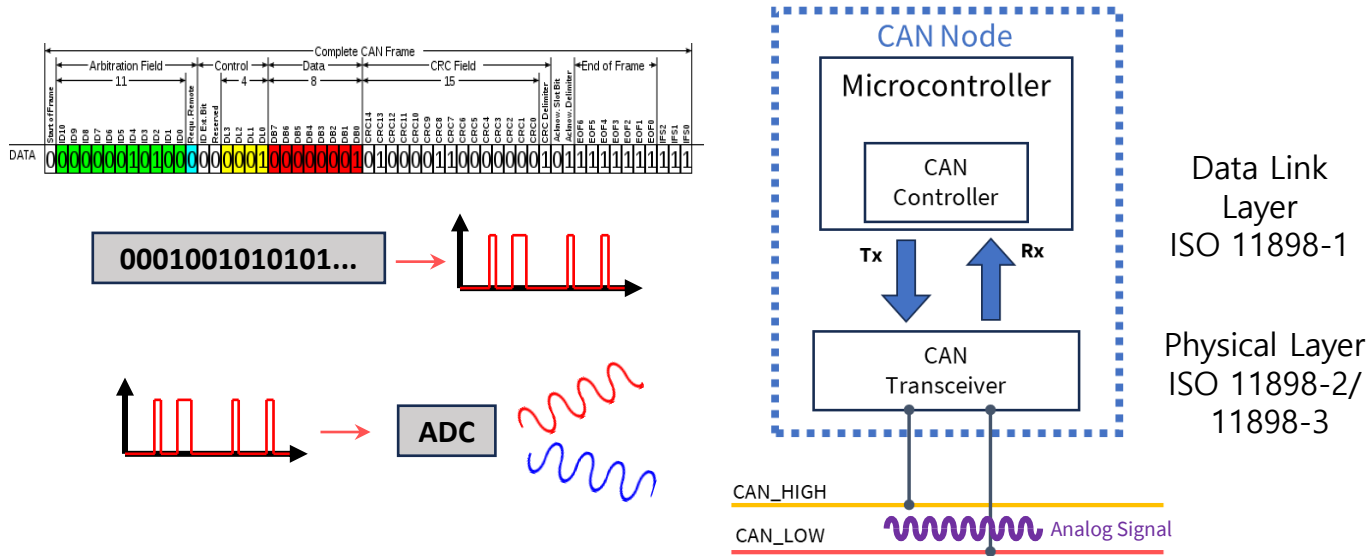
VCP-G

Support to CAN pin

- 따라서, 별도의 CAN Transceiver 장비를 사용해야, 보드간 CAN 통신을 수행할 수 있다.



CAN Message 전송 흐름



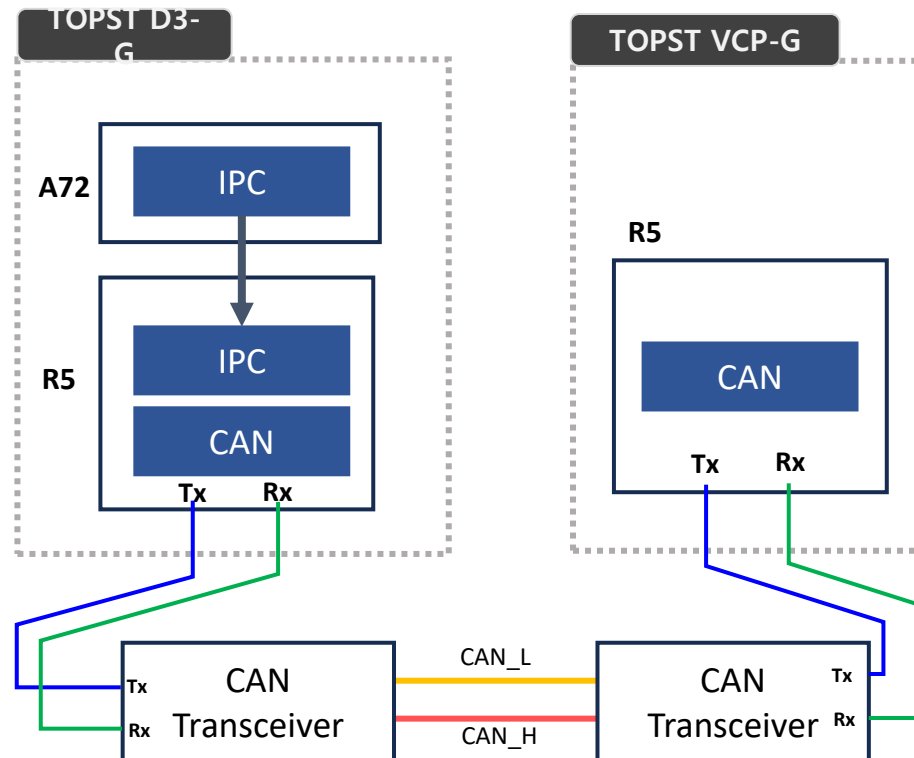
ISO 11898-1: CAN 데이터 링크 계층을 정의한다. CAN 프로토콜에서 데이터의 전송 및 오류 처리 기능 뿐만 아니라 표준 CAN 메시지의 구조와 프레임의 형식을 규정한다.

ISO 11898-2: 고속 CAN(CAN High-Speed)의 물리 계층을 정의한다. 차량 내 통신에서 주로 사용되며, 데이터 전송 속도는 최대 1 Mbps이다.

ISO 11898-3: 저속 CAN(CAN Low-Speed)의 물리 계층을 정의한다. 주로 차량의 간단한 네트워크 및 저속 통신에 사용되며 Fault Tolerant(내고장성, 결함 허용) 특징을 가지고 있다.

- CAN 통신과 연관된 동작을 하는 장치 : CAN controller
 - CAN controller 는 MCU 내부에 마련되어 있음
 - CAN 통신의 프로토콜을 따라서 실제로 CAN 통신을 할 수 있도록 지원
- CAN 컨트롤러에서 링크 계층 표준을 만족하며, 데이터의 전송, 수신 및 프로토콜을 담당한다.
- CAN Transceiver(**T**ransmitter + **R**eceiver)
 - 디지털 신호,아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
 - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
 - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

TOPST 보드간 CAN 메시지 전송 흐름

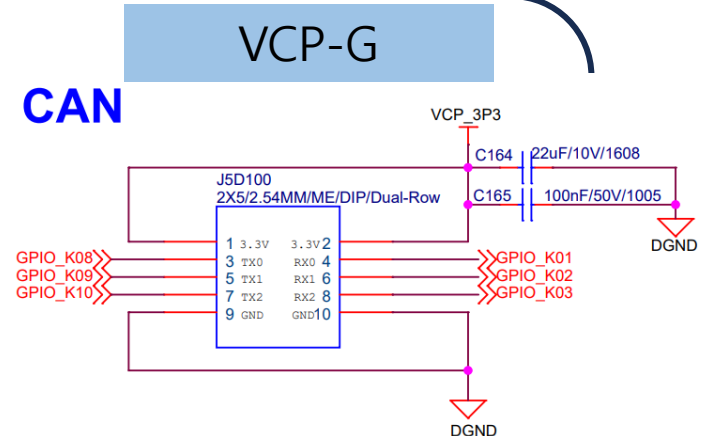
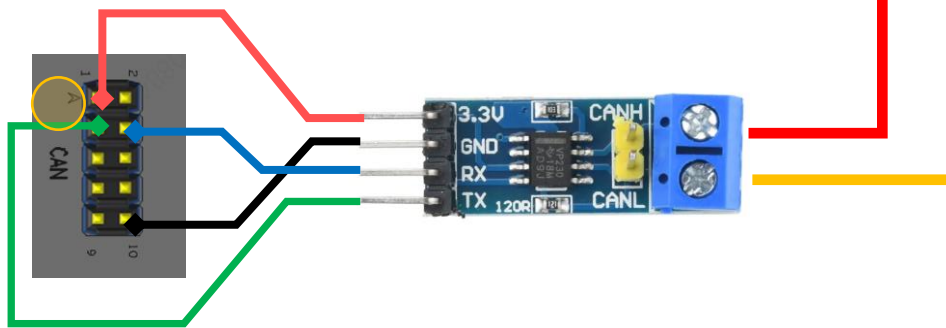
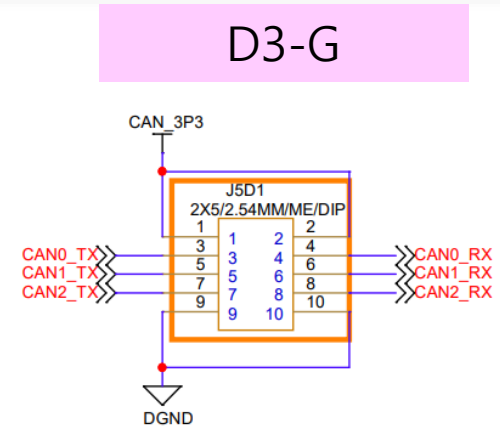
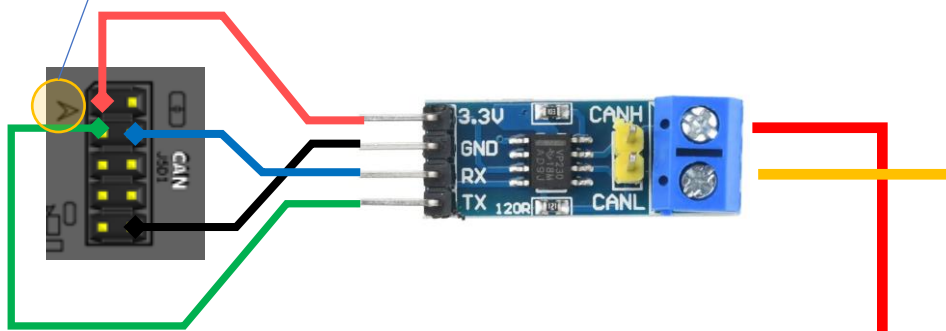


- 따라서 위와 같은 구조로 실습 수행
- D3-G [A72 >> R5] -> VCP-G 순으로 데이터 전송

실습을 위한 보드 연결

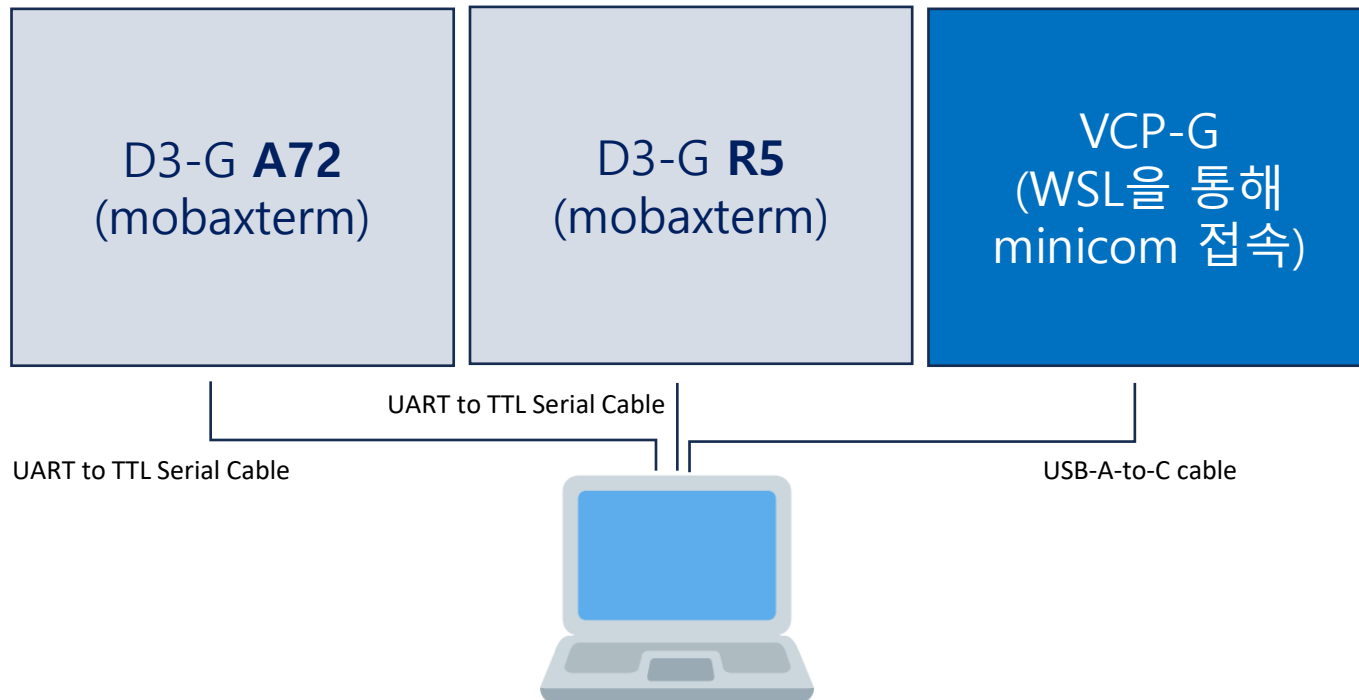
두 보드간 연결을 위해 아래와 같이 회로를 결선한다.

화살표 표시 부분이 1번핀



실습환경 세팅

- D3-G 보드와 VCP-G 보드와 UART 통신 세팅을 진행한다.



- A72 터미널내에서 IPC 명령어를 통해 R5 터미널과 VCP-G 터미널 화면을 관찰할 수 있다.
 - `$ cd motrex_education`

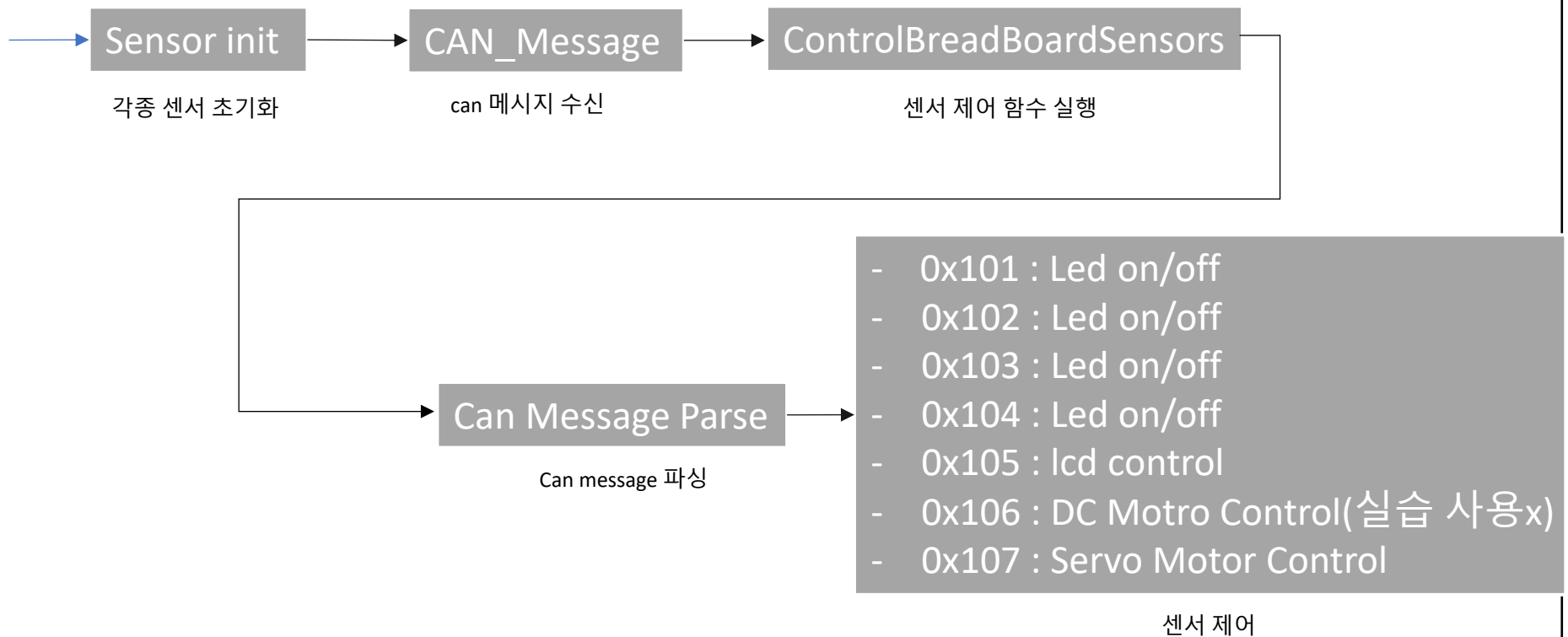
실습환경 세팅

- A72 IPC_Example 실행 커맨드 리스트 예제

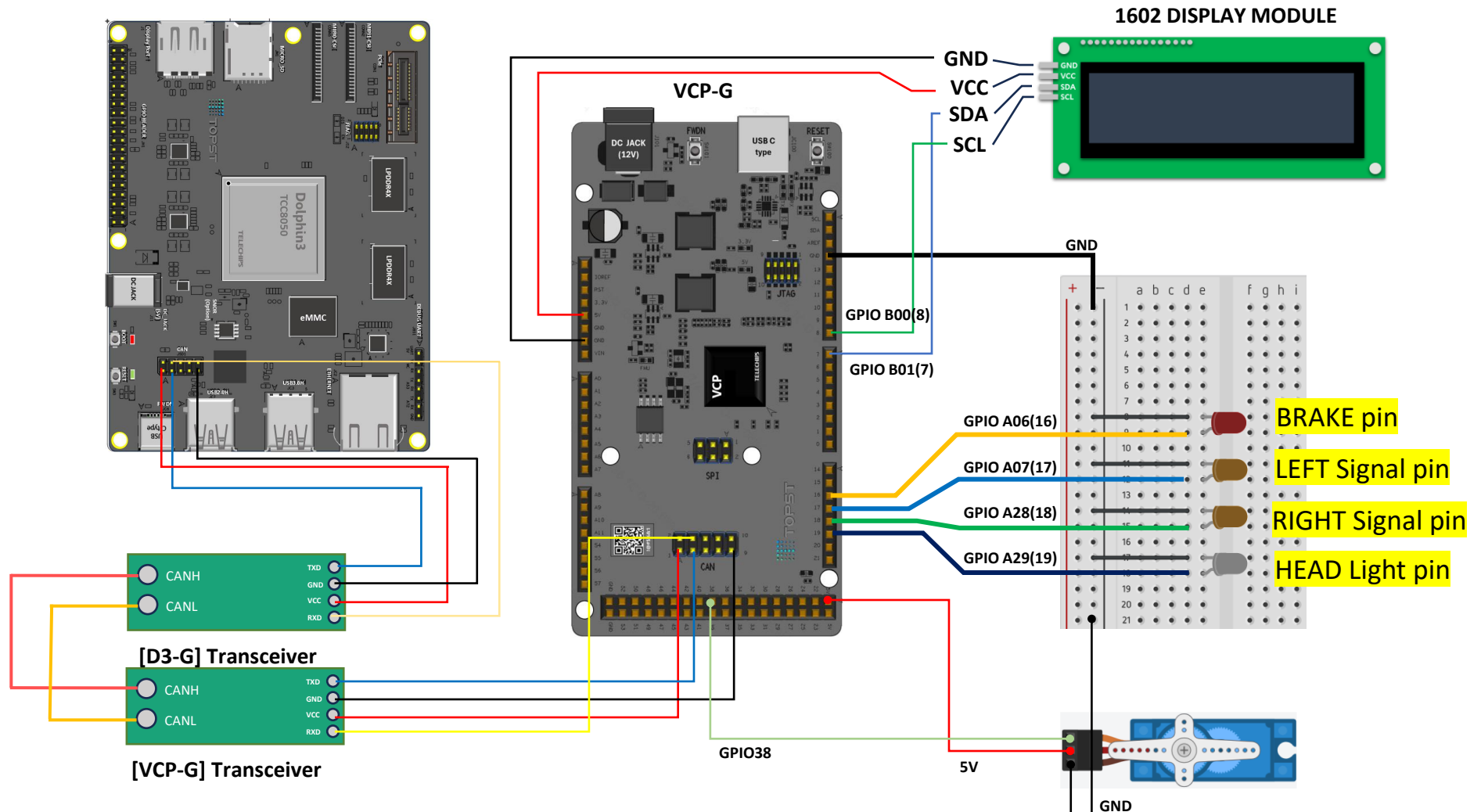
No.	Scenario	IPC Command
1	Brake Signal	\$ sudo python3 IPC_Example.py snd --canID 0x101 --sndDataHex 01 \$ sudo python3 IPC_Example.py snd --canID 0x101 --sndDataHex 02
2	Direction Signal	\$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 0101 \$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 0102 \$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 0201 \$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 0202
3	Emergency Signal	\$ sudo python3 IPC_Example.py snd --canID 0x103 --sndDataHex 01
4	HeadLight Signal	\$ sudo python3 IPC_Example.py snd --canID 0x104 --sndDataHex 01 \$ sudo python3 IPC_Example.py snd --canID 0x104 --sndDataHex 02
5	Fuel Status Signal	\$ sudo python3 IPC_Example.py snd --canID 0x105 --sndDataHex 33
6	Motor Speed Signal	\$ sudo python3 IPC_Example.py snd --canID 0x106 --sndDataHex 33
7	Motor Direction Signal	\$ sudo python3 IPC_Example.py snd --canID 0x107 --sndDataHex 7F

TOPST 통합 제어 시스템 코드 리뷰(VCP-G)

• 동작 시나리오(VCP-G)



VCP-G 와 모형차 Kit 회로 연결



실습 결과

```

topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd
--canID 0x103 --sndDataHex 01
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0103
channel_bitmask: 0x01 (channels: 0)
tx_only channel bitmask: 0x00 (channels: None)
canID: 0x0103
sndData: b'\x01'
sndDataHex: 01
sndDataLength: 1
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$

(D)[CAN ][CAN_Edu_SetSendingData:1567]
ch_bitmask: 0x1, tx_only_bitmask: 0x0, can_id: 0x101
(D)[CAN ][CAN_Edu_SetSendingData:1583] [CH] : 0x0, [ID] (D)[MBOX ][MBOX_Dev
Qget:1314] empty for ch[3] siDevId[2]
: 0x101, [DATA SIZE] : 1 (5), [DATA] :
(D)[CAN ][CAN_Edu_SetSendingData:1586] 0x02
(D)[CAN ][CAN_Edu_SetSendingData:1591]
(D)[CAN ][CAN_Edu_SetSendingData:1599] gNowMetaInfo.channel : 0x7
(D)[CAN ][CAN_Edu_DemoSend:1332] canTxMsg => id: 257, isFE: 0, extID: 0, le
n: 1, data:
(D)[CAN ][CAN_Edu_DemoSend:1345]
ret: 0
(D)[CAN ][CAN_Edu_DemoSend:1369]
CAN_SEND ch: 0 id: 257 mDataLength:1 (1:10)
(D)[CAN ][CAN_Edu_DemoSend:1375] Successful transmission.
(D)[TIMER][TIMER_Disable:594] Channel (1) is disabled
[PMIO][PMIO_STR_SetTimer:1727] pmio Set Timer disable, reason clear
[PMIO][PMIO_STR_TimeoutInterruptHandler:1750] Timeout : ACC ON signal
[PMIO][PMIO_STR_TimeoutInterruptHandler:1751] Request : STR mode
(D)[MBOX ][MBOX_DevQget:1314] empty for ch[3] siDevId[2]
(D)[MBOX ][MBOX_DevQget:1314] empty for ch[3] siDevId[2]
(D)[CAN ][CAN_Edu_SetSendingData:1567]
ch_bitmask: 0x1, tx_only_bitmask: 0x0, can_id: 0x103
(D)[CAN ][CAN_Edu_SetSendingData:1583] [CH] : 0x0, [ID] (D)[MBOX ][MBOX_Dev
Qget:1314] empty for ch[3] siDevId[2]
: 0x103, [DATA SIZE] : 1 (5), [DATA] :
(D)[CAN ][CAN_Edu_SetSendingData:1586] 0x01
(D)[CAN ][CAN_Edu_SetSendingData:1591]

[CAN DEMO]
< Channel 0 Received Message Information >
*****
[ID] : 0x101, [DATA SIZE] : 1, [DATA] :
0x01
*****
Brake Light ON
[CAN DEMO] No message
[CAN DEMO]
< Channel 0 Received Message Information >
*****
[ID] : 0x101, [DATA SIZE] : 1, [DATA] :
0x02
*****
Brake Light OFF
[CAN DEMO] No message
[CAN DEMO]
< Channel 0 Received Message Information >
*****
[ID] : 0x101, [DATA SIZE] : 1, [DATA] :
0x02
*****
Brake Light OFF
[CAN DEMO] No message
[CAN DEMO]
< Channel 0 Received Message Information >
*****
[ID] : 0x103, [DATA SIZE] : 1, [DATA] :
0x01
*****
Emergency Signal ON<-->OFF
[ControlBreadBoardSensors][204] undefined can id !!!
[CAN DEMO] No message
  
```

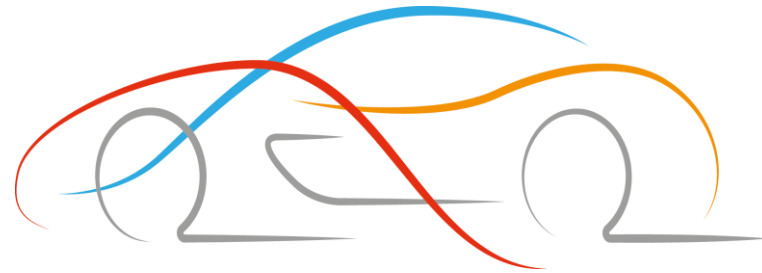
D3-G A72 (IPC_Example)



D3-G R5



VCP-G



Thank You